

Deep Learning for Computer Vision (Discriminative way)

Andrii Liubonko
2025

Course structure

Module 1. Basic image processing - Oles Dobosevych

Module 2. Learning Discriminative Models - Andrii Liubonko

Module 3. Image Segmentation, Image Correspondence -
Maksym Davydov

Module 4. Metric Learning, Representation Learning and Context
Importance, Tracking - Igor Krashenyi

Module 5. Learning Generative Models - Andrii Liubonko

Logistics (this module)

6 lectures

homework:

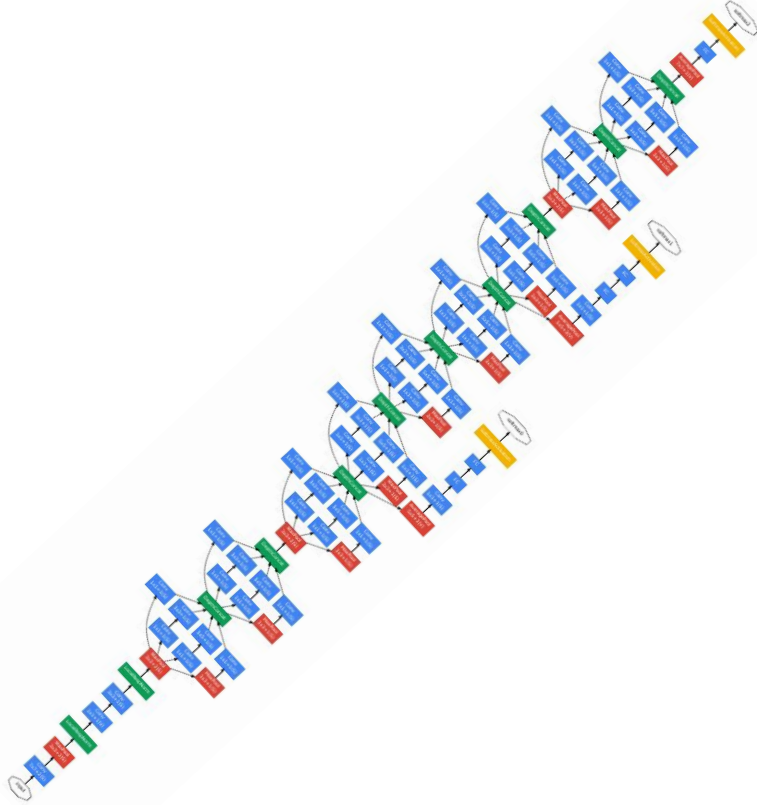
- assignment 1 : tutorials notebooks 50%
- assignment 2 : mini-project 50%

deadline date: *TBD*

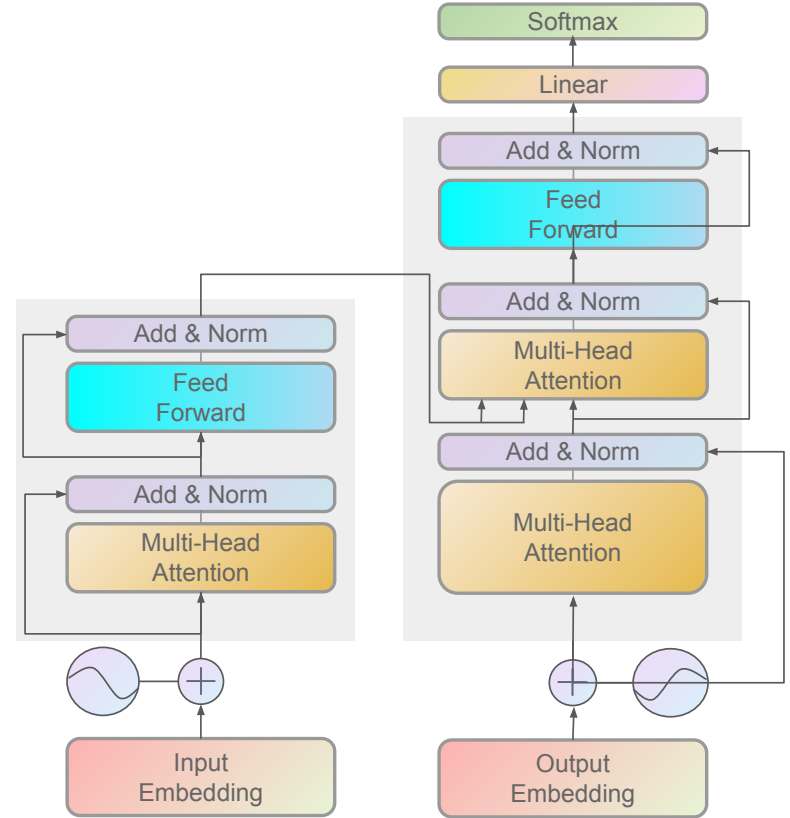
course repo:

<https://github.com/lyubonko/ucu2025cv>

CV DL fundamentals



Convolutional Neural Networks (CNN)



Transformer-based

Content of today's lecture

- Attention Intro
 - Seq2seq Attention
 - Self-Attention
 - Transformer
- Attention in CV
 - SE
 - ViT
 - latest developments

“NLP”

Seq2Seq

- [Cho et al. \(2014\)](#)
- [Sutskever et al. \(2014\)](#)

Align & Translate

- [Bahdanau et al. \(2015\)](#)
- [Luong et al. \(2015\)](#)

Transformer

- [Vaswani et al. \(2017\)](#)

GPT

- [OpenAI \(2018\)](#)

GPT-3

- [Brown et al. \(2020\)](#)

LLAMA

- [Meta \(2023\)](#)

GPT-5

- [OpenAI \(2025\)](#)

BERT

- [Devlin et al. \(2018\)](#)

DeepSeek

- [DeepSeek-AI \(2024\)](#)

“CV”

Visual attention

- [Xu et al. \(2015\)](#)

SE networks

- [Hu et al. \(2018\)](#)

DETR

- [Carion et al. \(2020\)](#)

MAE

- [He et al. \(2021\)](#)

ViT

- [Dosovitskiy et al. \(2020\)](#)

DINOv1

- [Caron et al. \(2021\)](#)

SAMv1

- [Meta \(2023\)](#)

DINOv3

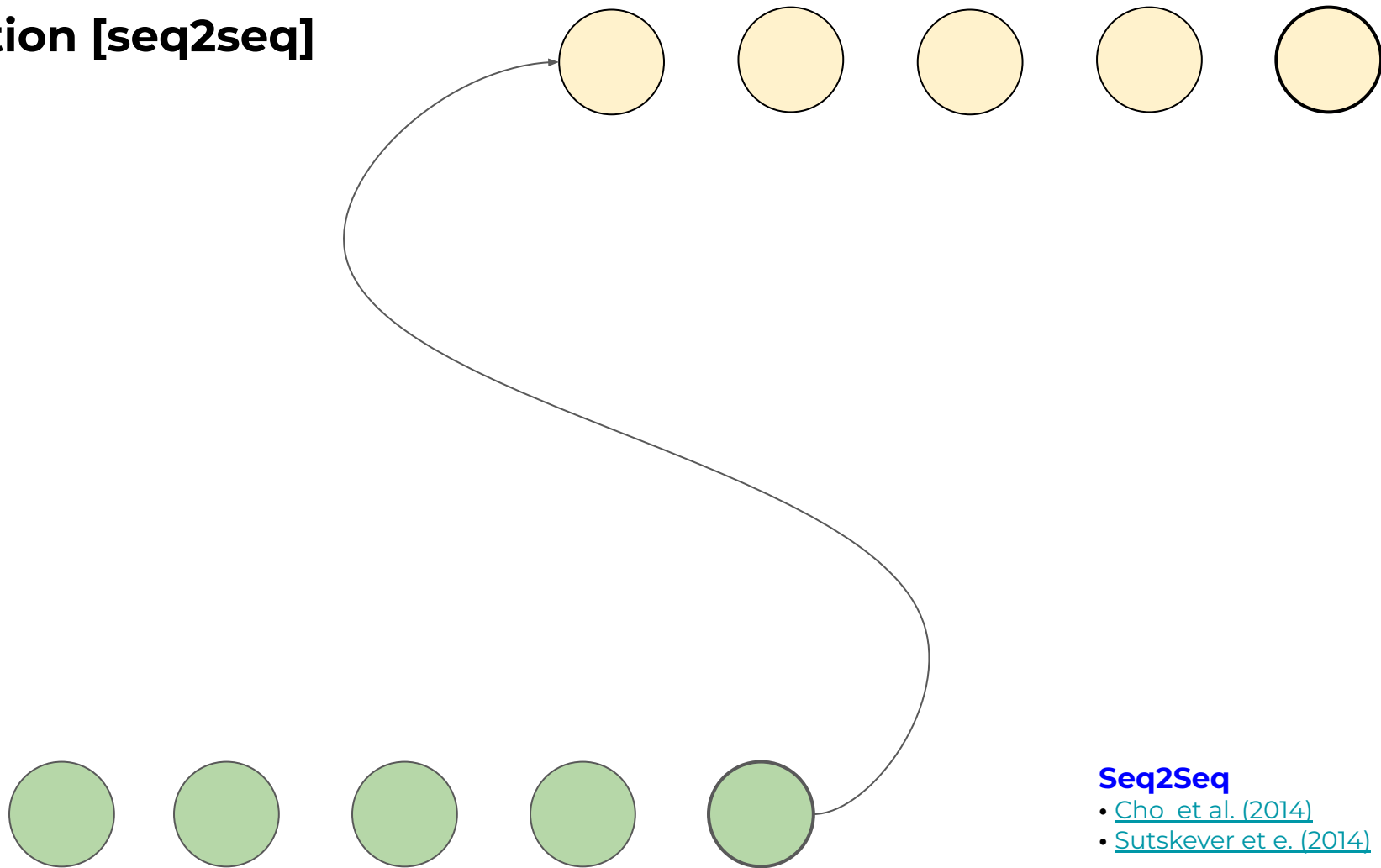
- [Meta \(2025\)](#)

Attention [intro]



In a nutshell, attention in deep learning can be broadly interpreted as a vector of importance weights: in order to predict or infer one element, such as a pixel in an image or a word in a sentence, we estimate using the attention vector how strongly it is correlated with (or “attends to”) other elements

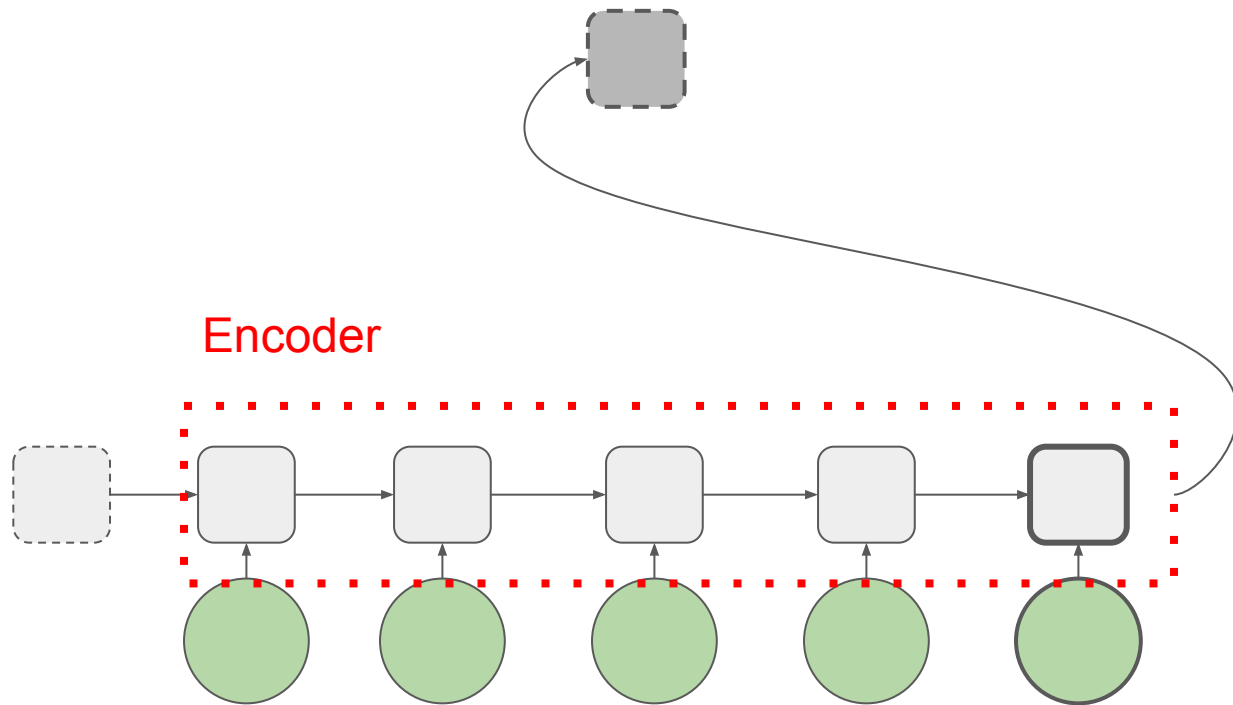
Attention [seq2seq]



Seq2Seq

- [Cho et al. \(2014\)](#)
- [Sutskever et e. \(2014\)](#)

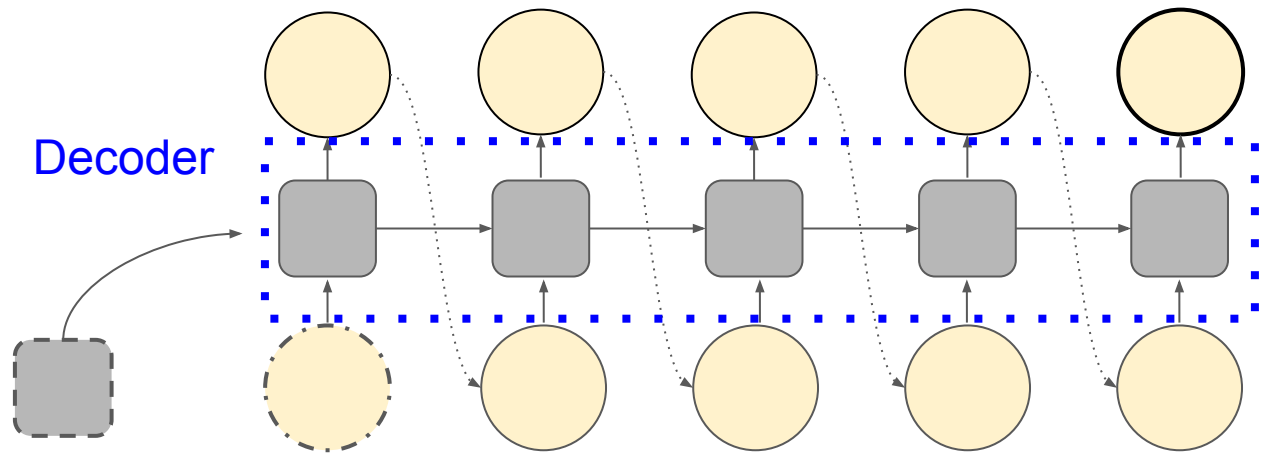
Attention [seq2seq]



Seq2Seq

- [Cho et al. \(2014\)](#)
- [Sutskever et al. \(2014\)](#)

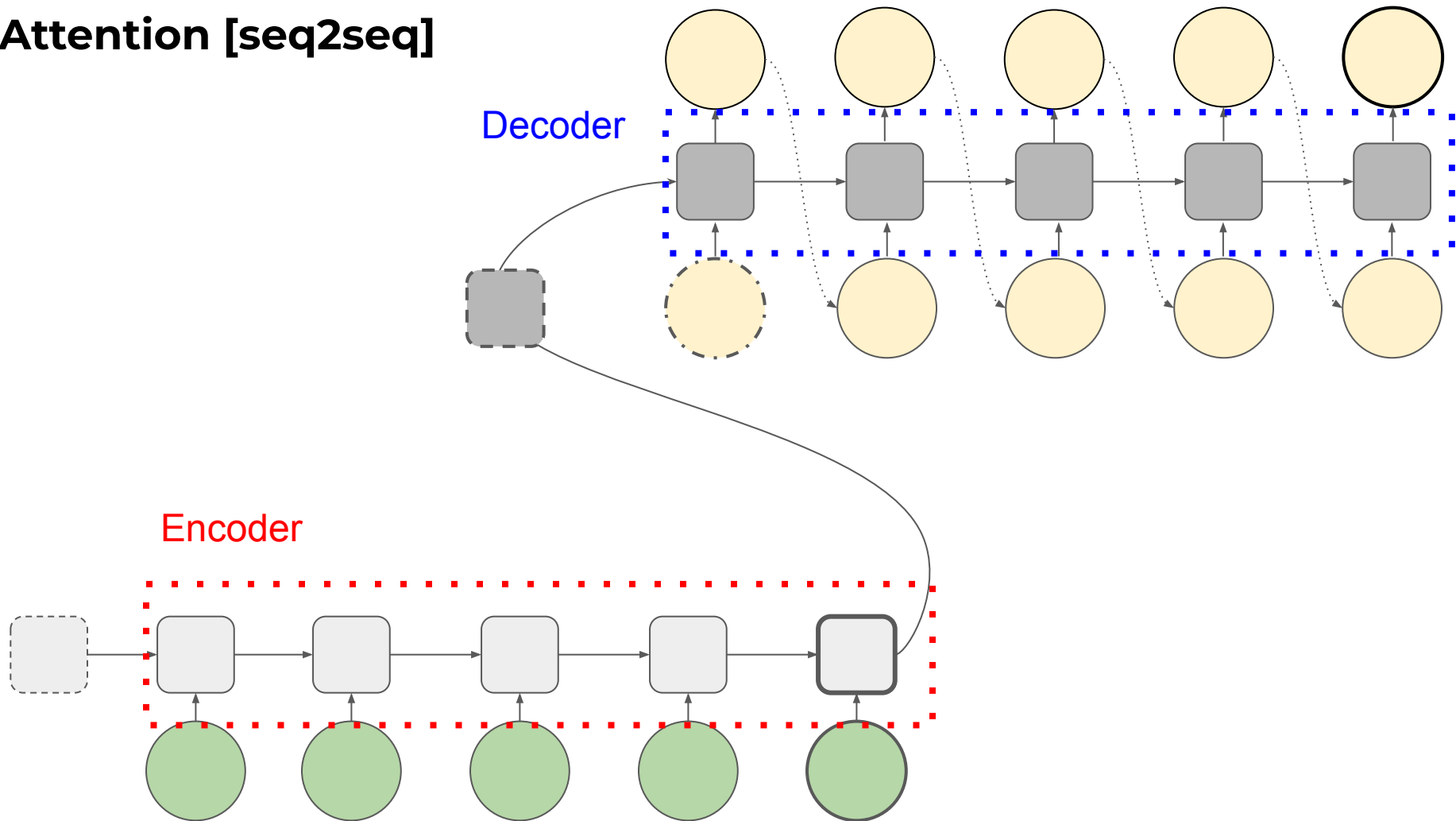
Attention [seq2seq]



Seq2Seq

- [Cho et al. \(2014\)](#)
- [Sutskever et al. \(2014\)](#)

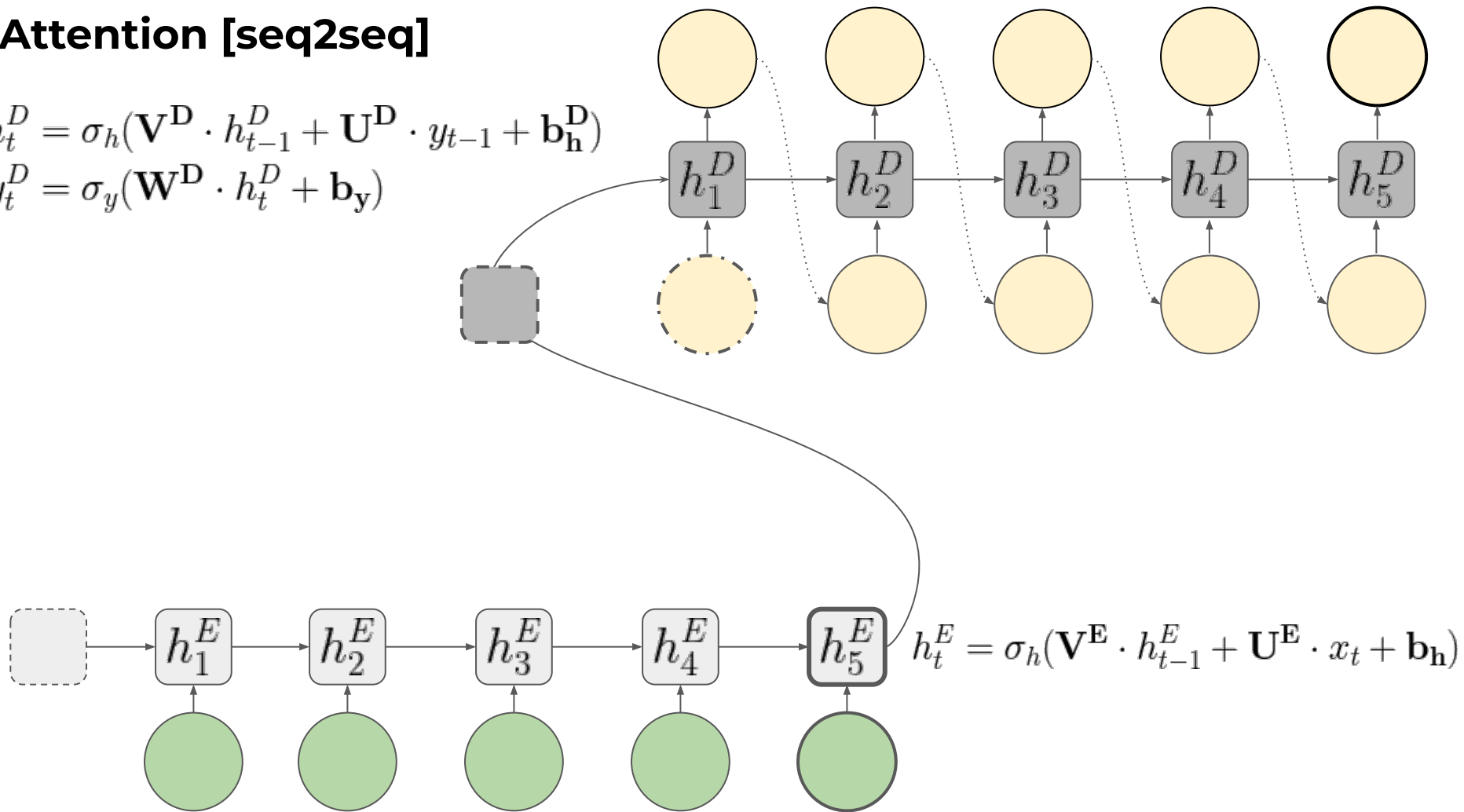
Attention [seq2seq]



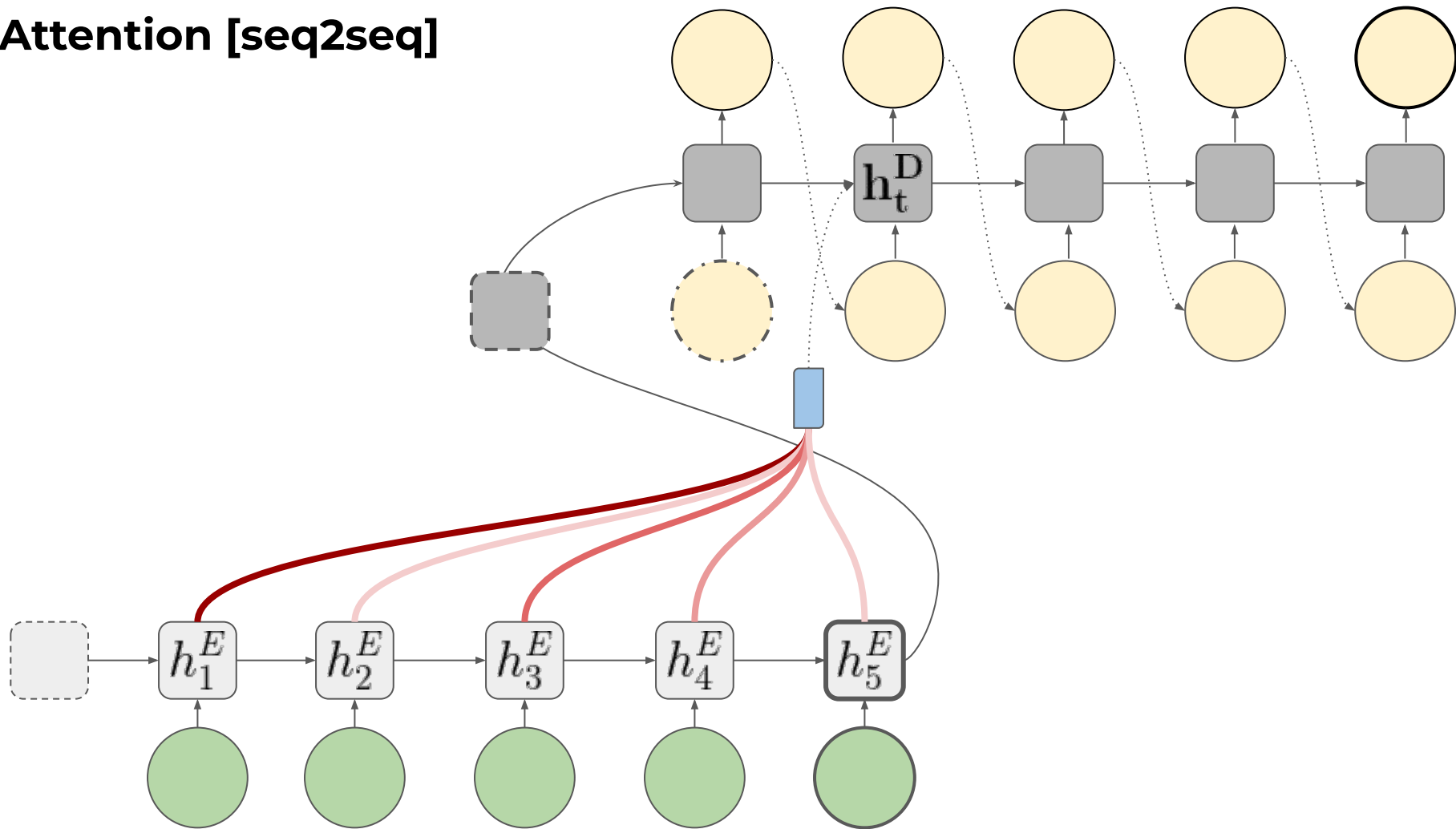
Attention [seq2seq]

$$h_t^D = \sigma_h(\mathbf{V}^D \cdot h_{t-1}^D + \mathbf{U}^D \cdot y_{t-1} + \mathbf{b}_h^D)$$

$$y_t^D = \sigma_y(\mathbf{W}^D \cdot h_t^D + \mathbf{b}_y)$$



Attention [seq2seq]



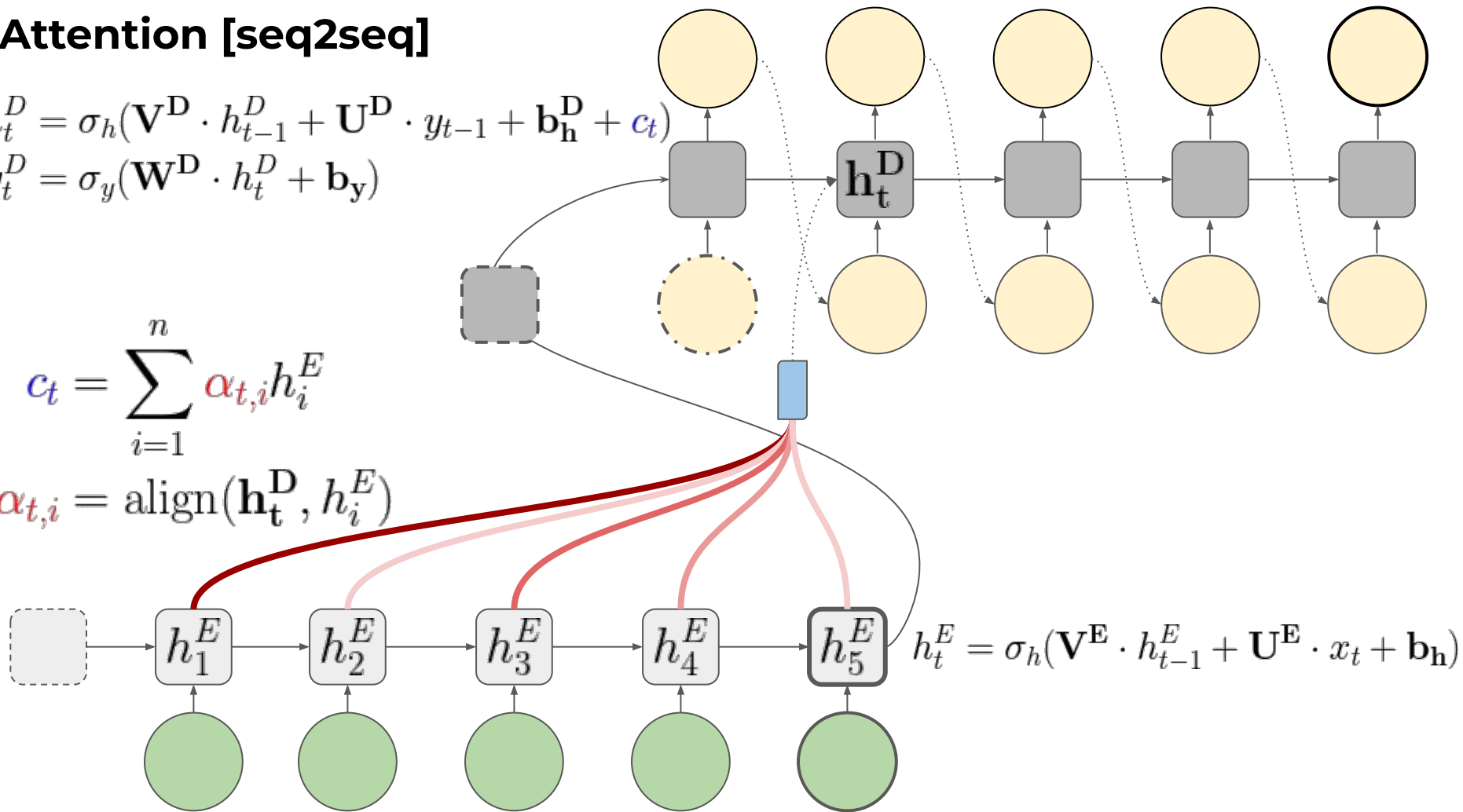
Attention [seq2seq]

$$h_t^D = \sigma_h(\mathbf{V}^D \cdot h_{t-1}^D + \mathbf{U}^D \cdot y_{t-1} + \mathbf{b}_h^D + \mathbf{c}_t)$$

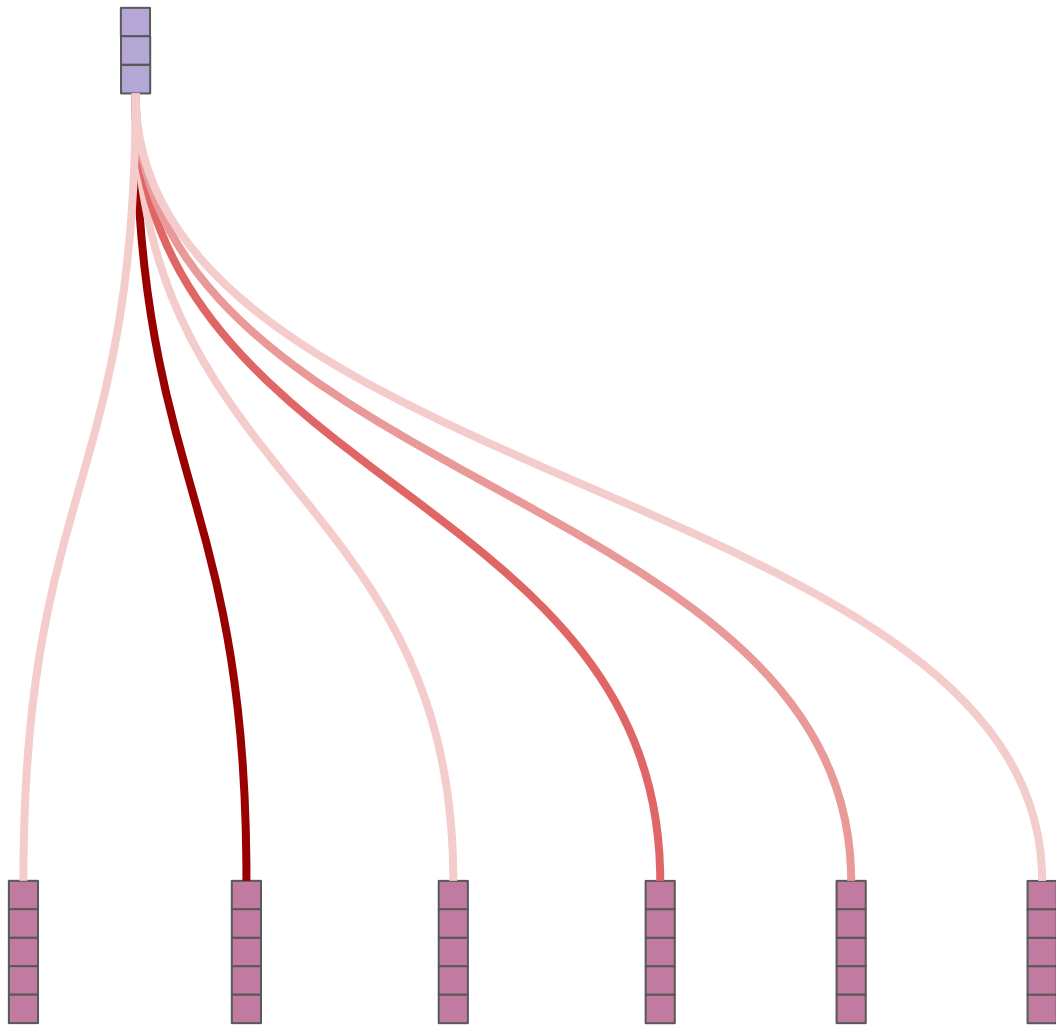
$$y_t^D = \sigma_y(\mathbf{W}^D \cdot h_t^D + \mathbf{b}_y)$$

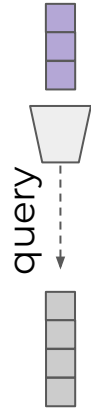
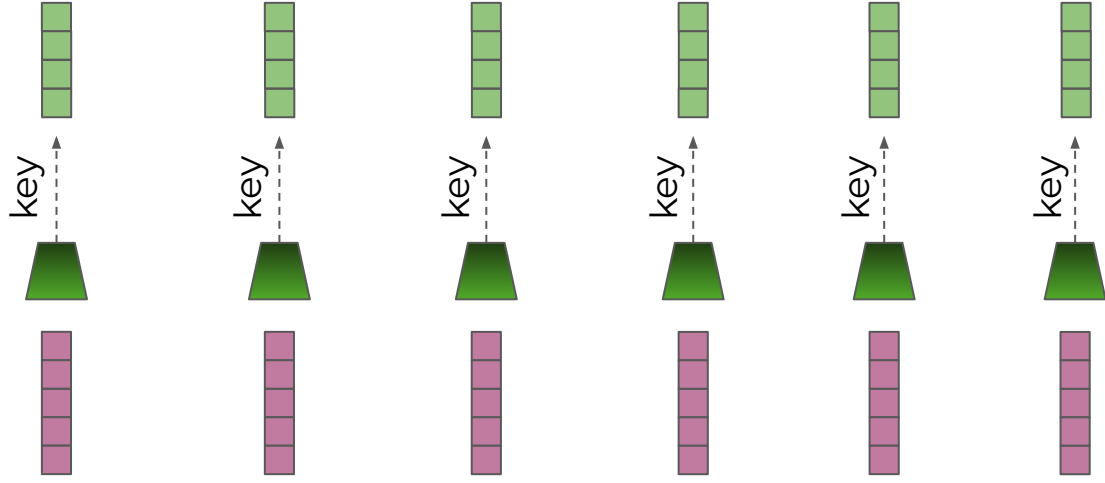
$$\mathbf{c}_t = \sum_{i=1}^n \alpha_{t,i} h_i^E$$

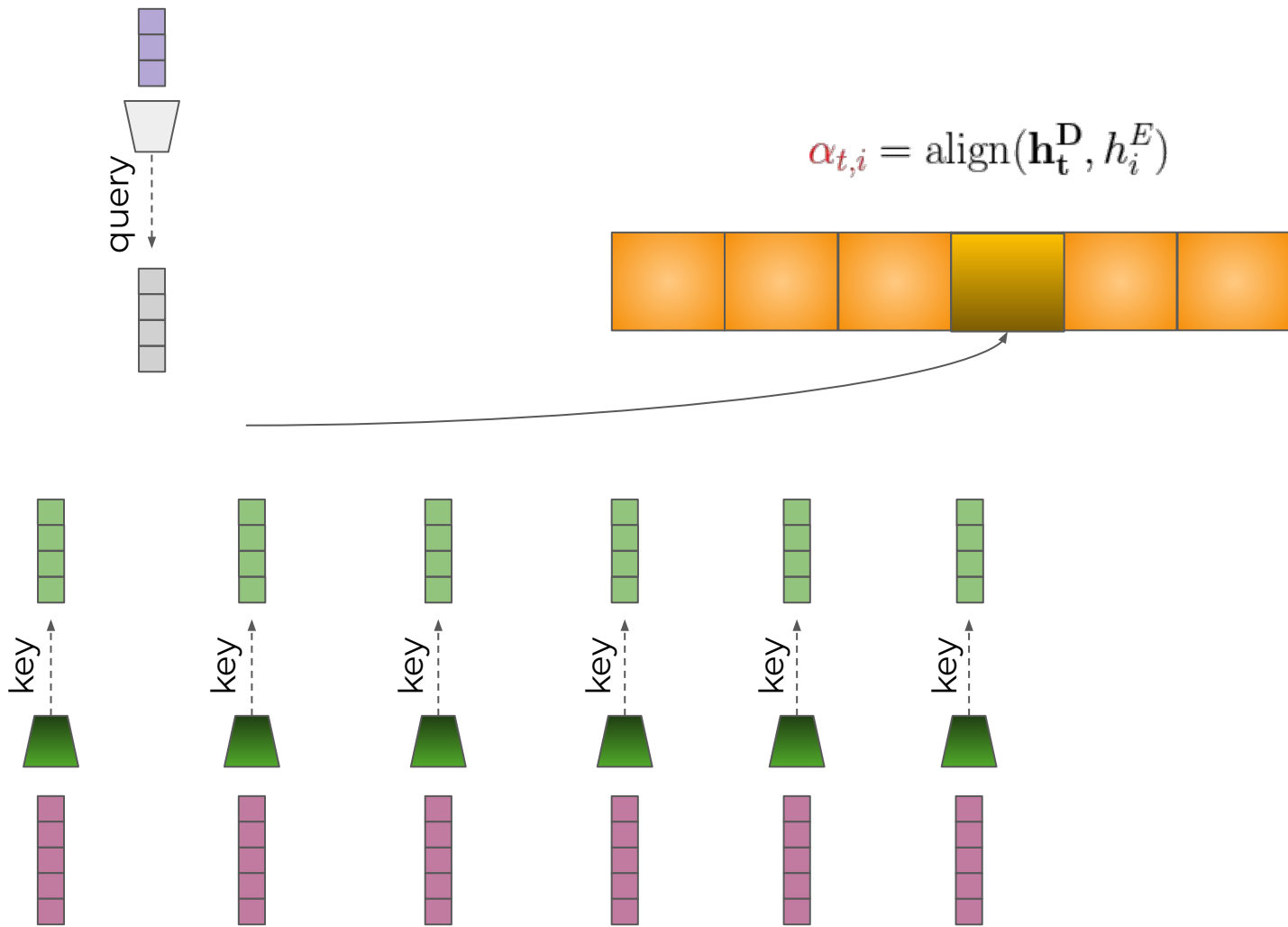
$$\alpha_{t,i} = \text{align}(\mathbf{h}_t^D, h_i^E)$$



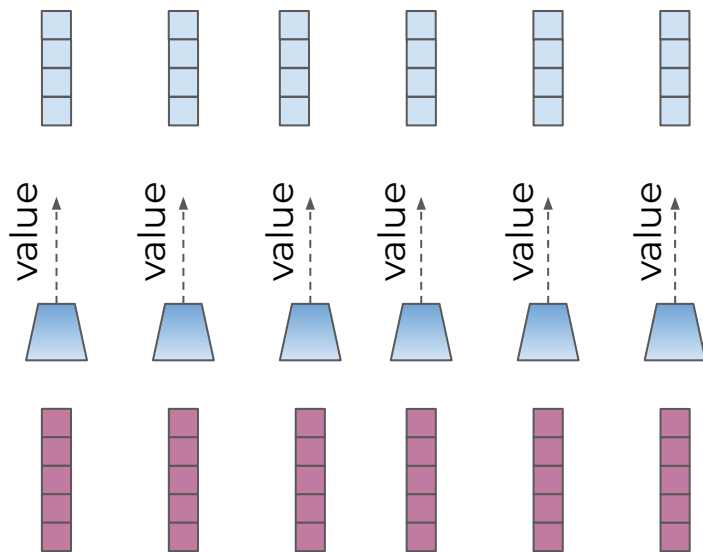
query-key-value abstraction







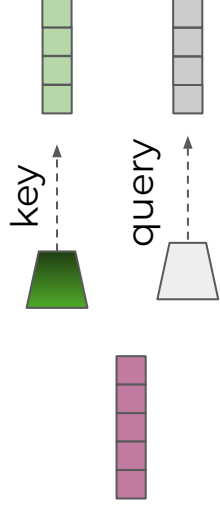
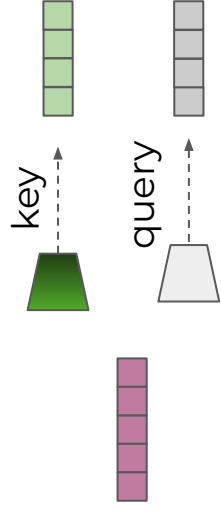
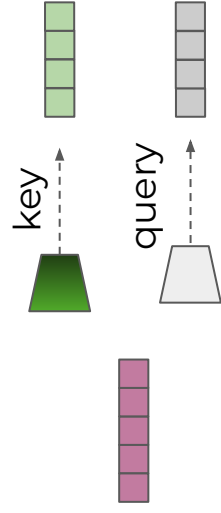
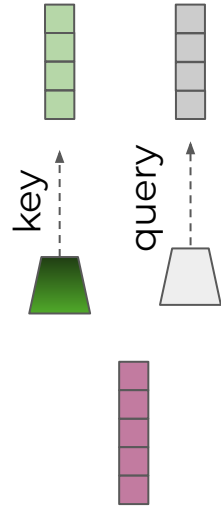
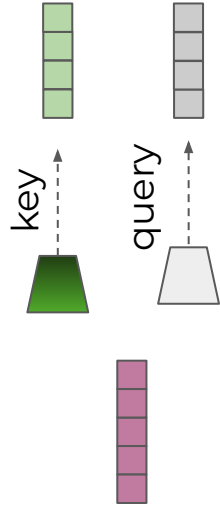
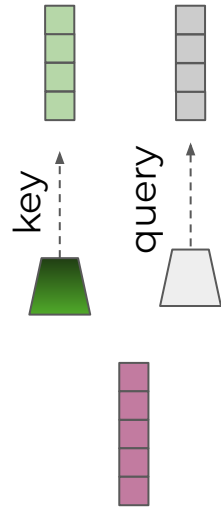
$$\alpha_{t,i} = \text{align}(\mathbf{h}_t^D, h_i^E)$$

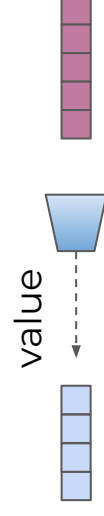
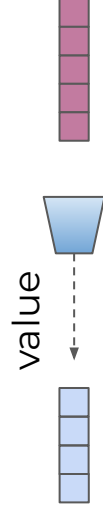
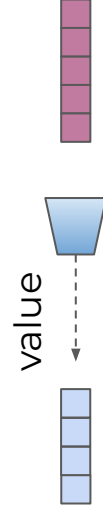
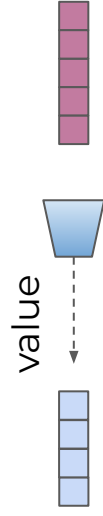
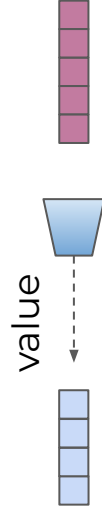
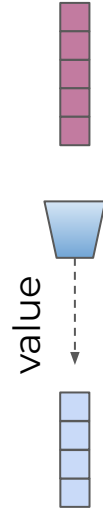


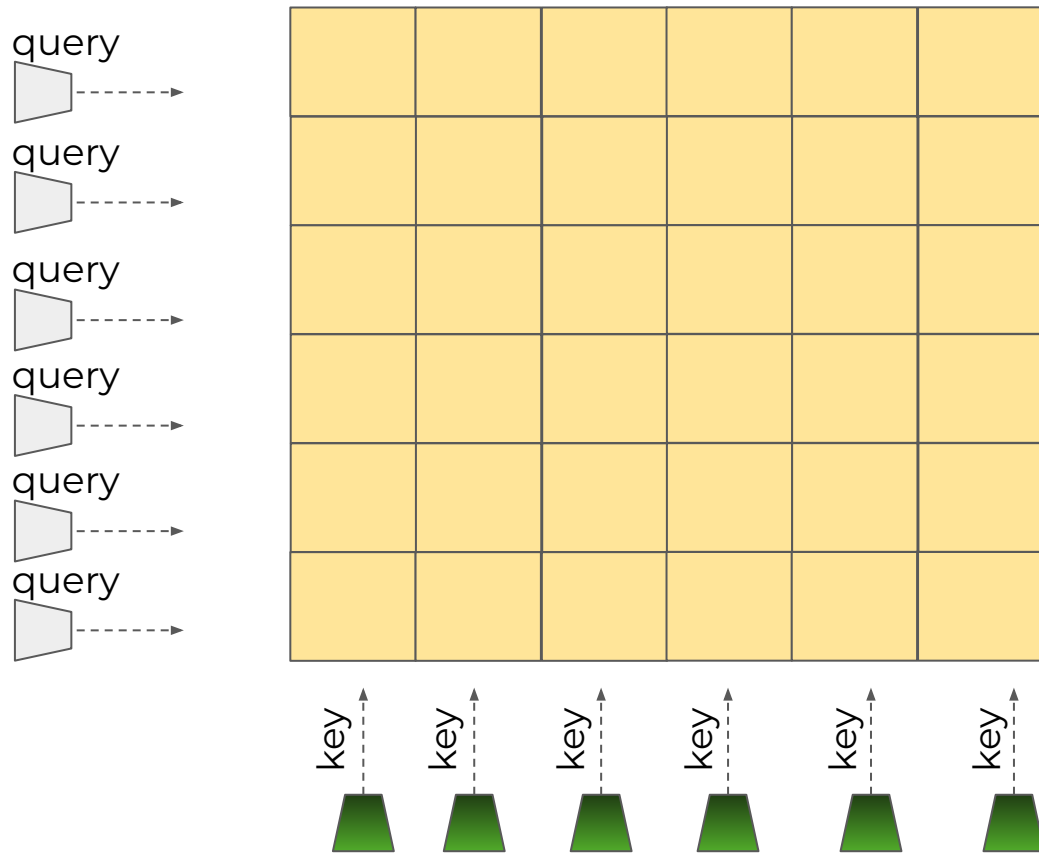
$$c_t = \sum_{i=1}^n \alpha_{t,i} h_i^E$$

Self-attention

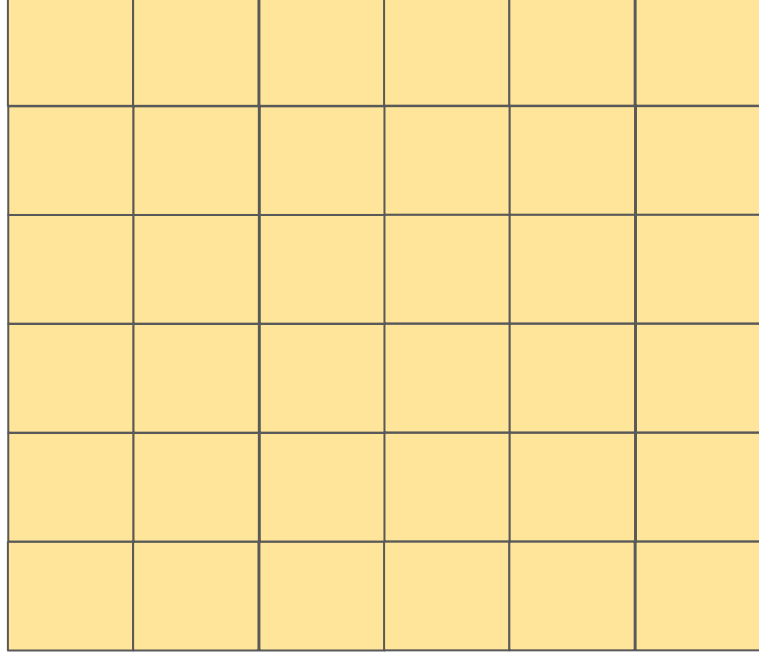








“values”



11	21	31			
12	22	32			
13	23	33			
14	24	34			
15	25	35			
16	26	36			

$$\begin{array}{c}
 \boxed{11} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{12} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{13} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{14} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{15} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{16} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} = \\
 \end{array}$$



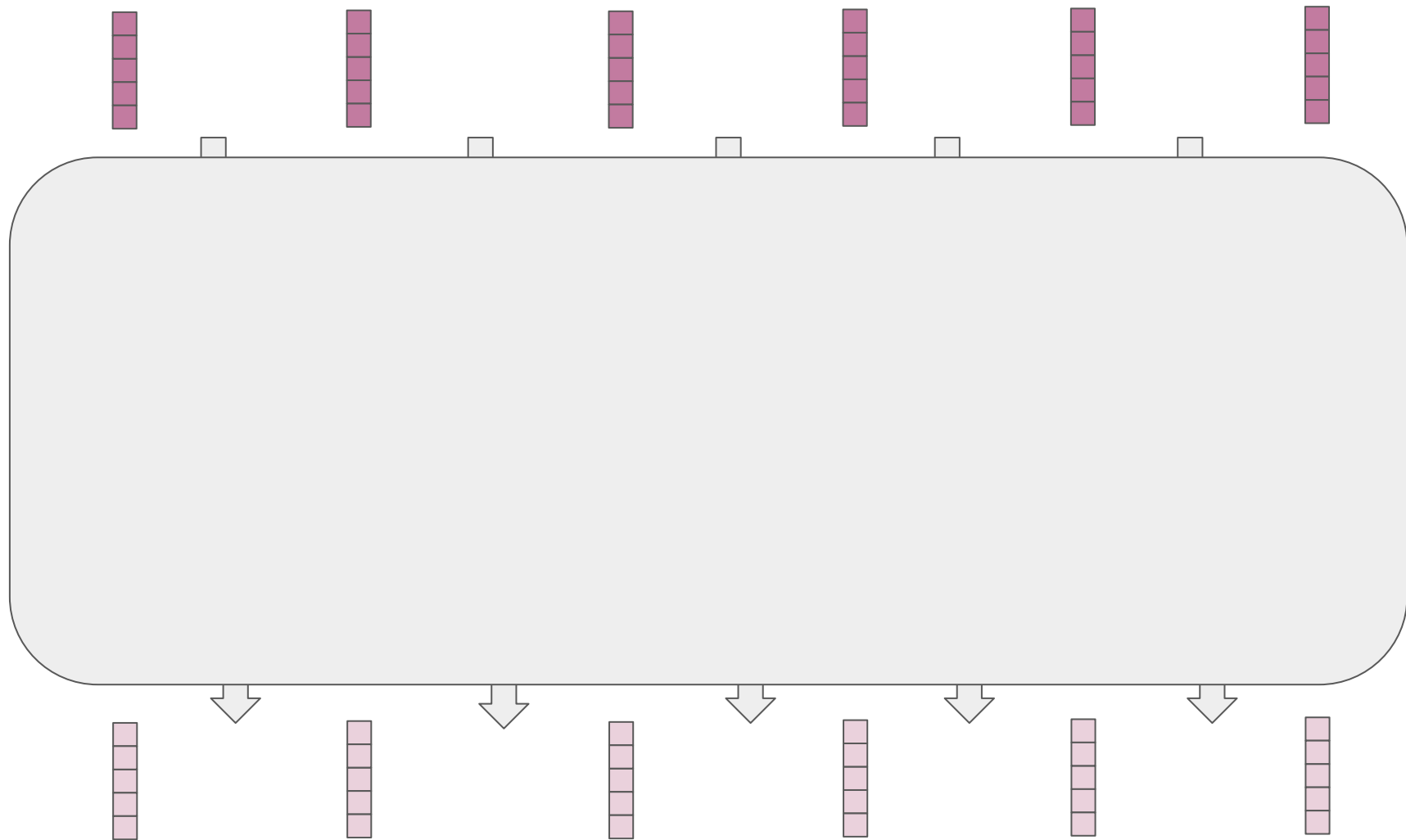
11	21	31			
12	22	32			
13	23	33			
14	24	34			
15	25	35			
16	26	36			

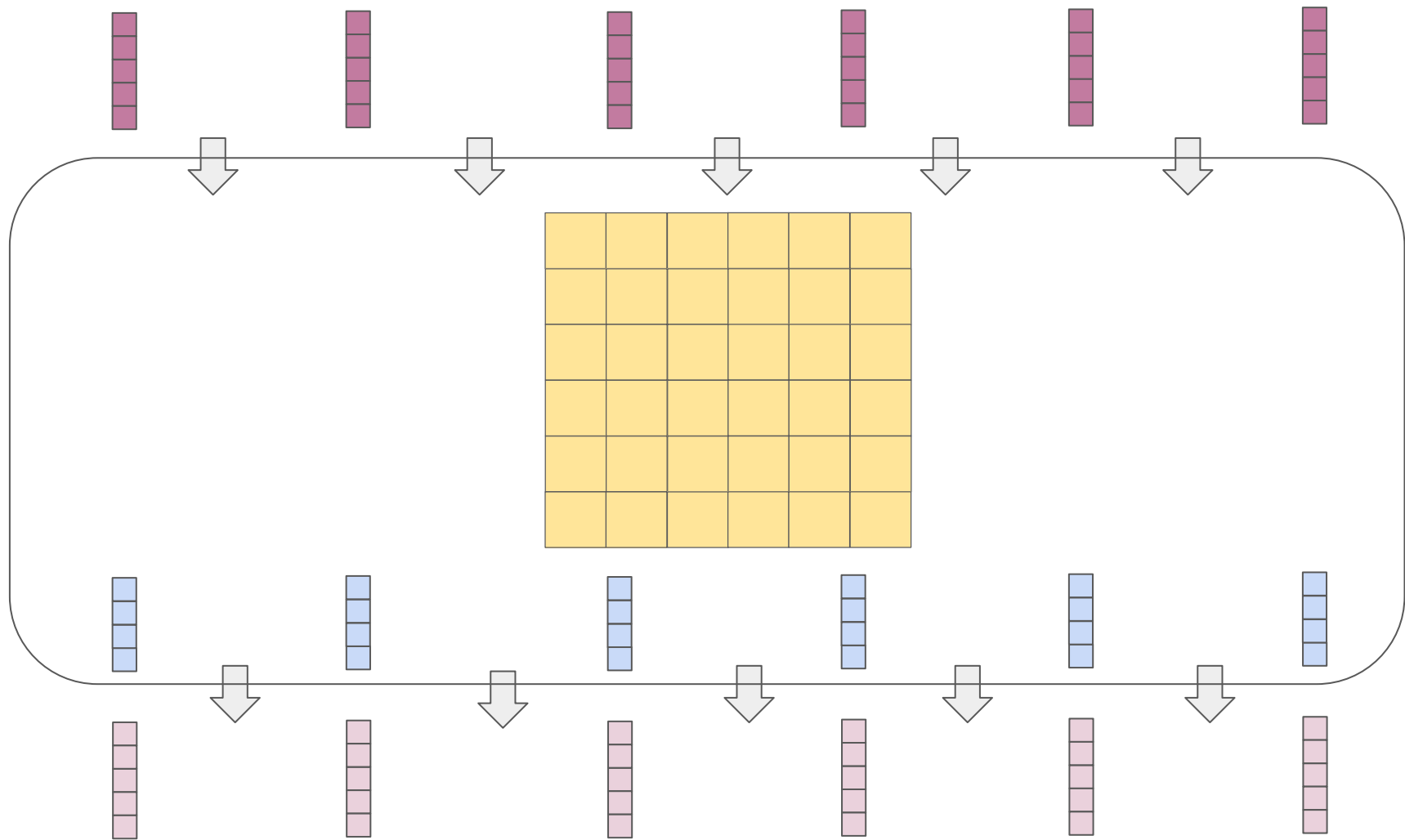
$$\begin{array}{c}
 \boxed{21} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{22} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{23} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{24} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{25} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{26} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} = \\
 \end{array}$$



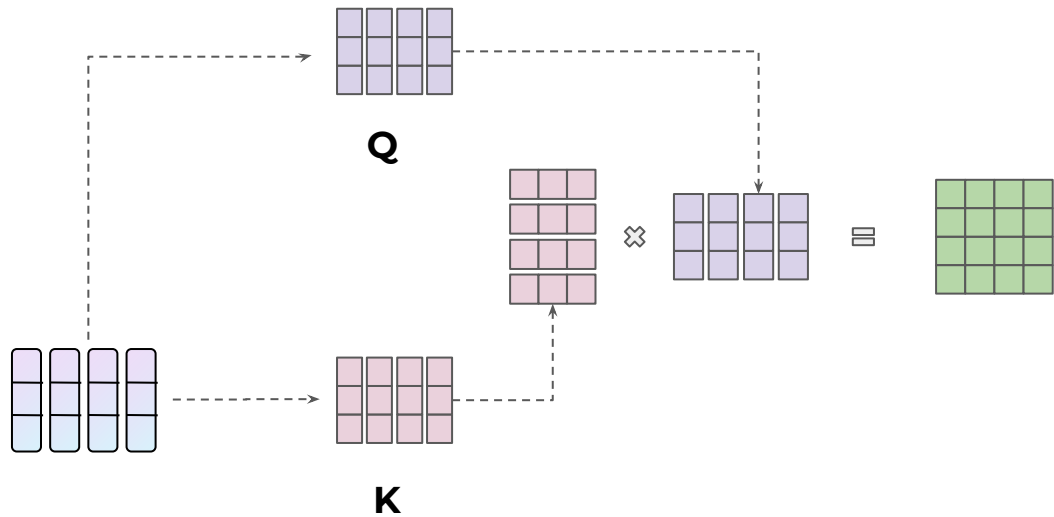
11	21	31			
12	22	32			
13	23	33			
14	24	34			
15	25	35			
16	26	36			

$$\begin{array}{c}
 \boxed{31} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{32} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{33} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{34} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{35} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} + \boxed{36} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} = \\
 \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array}
 \end{array}$$

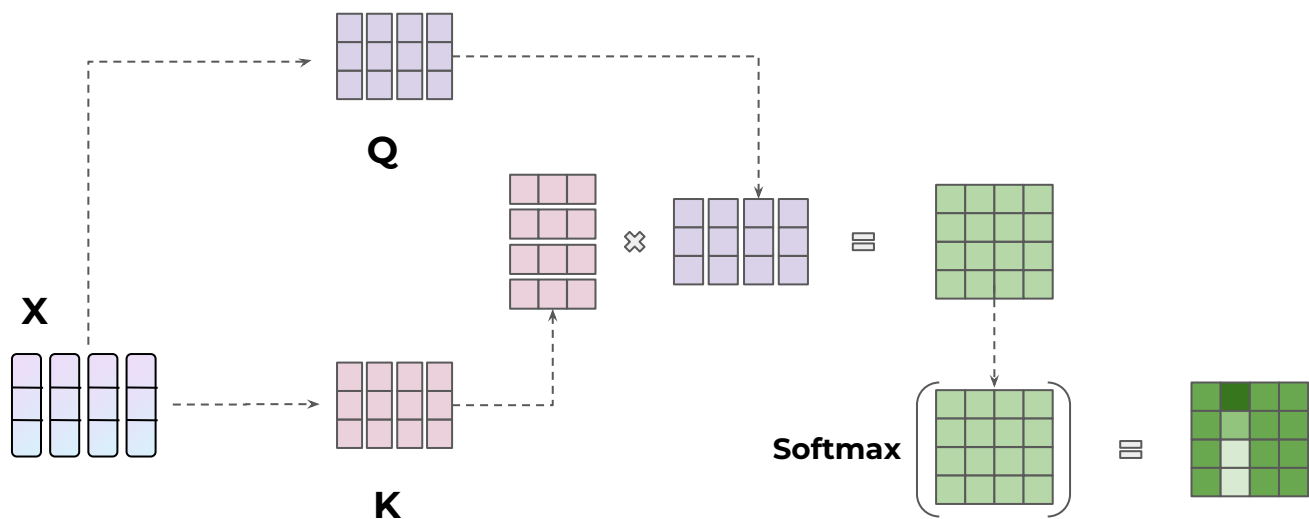




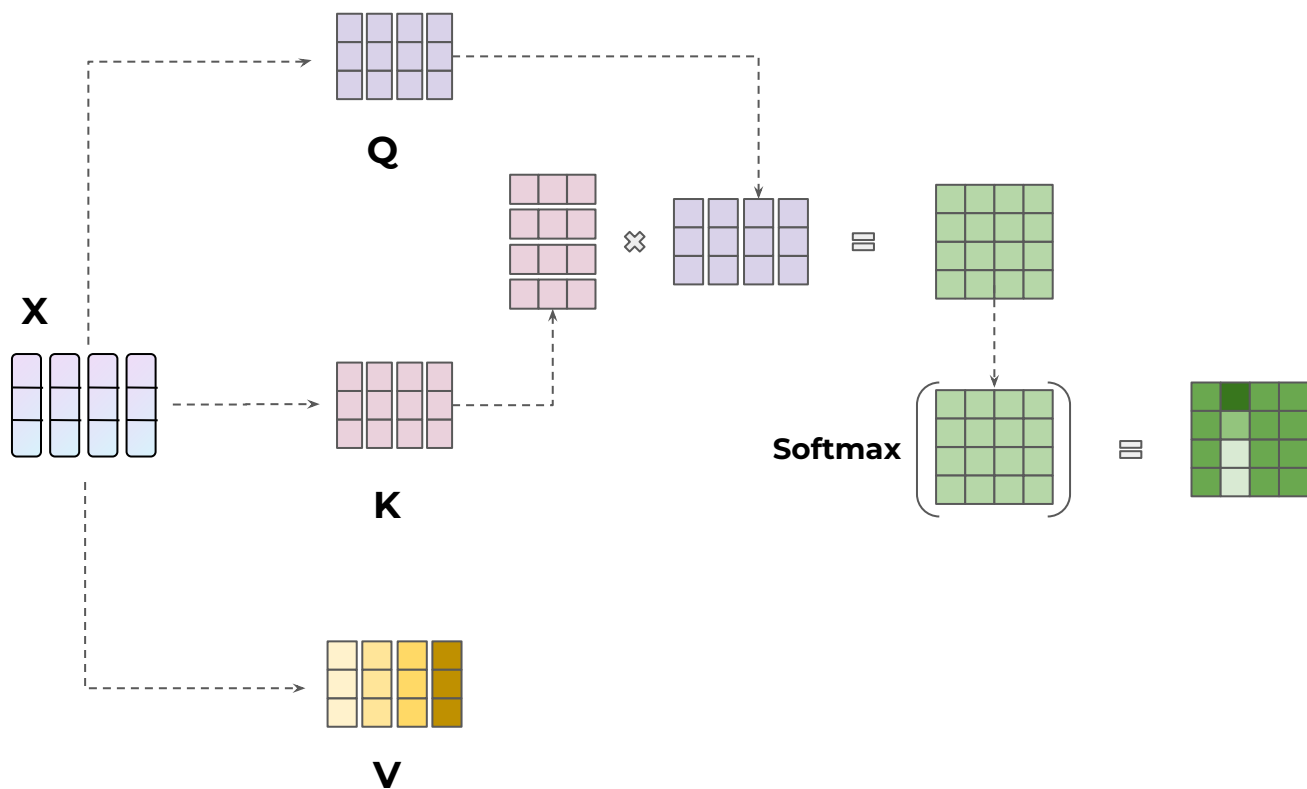
Self-Attention



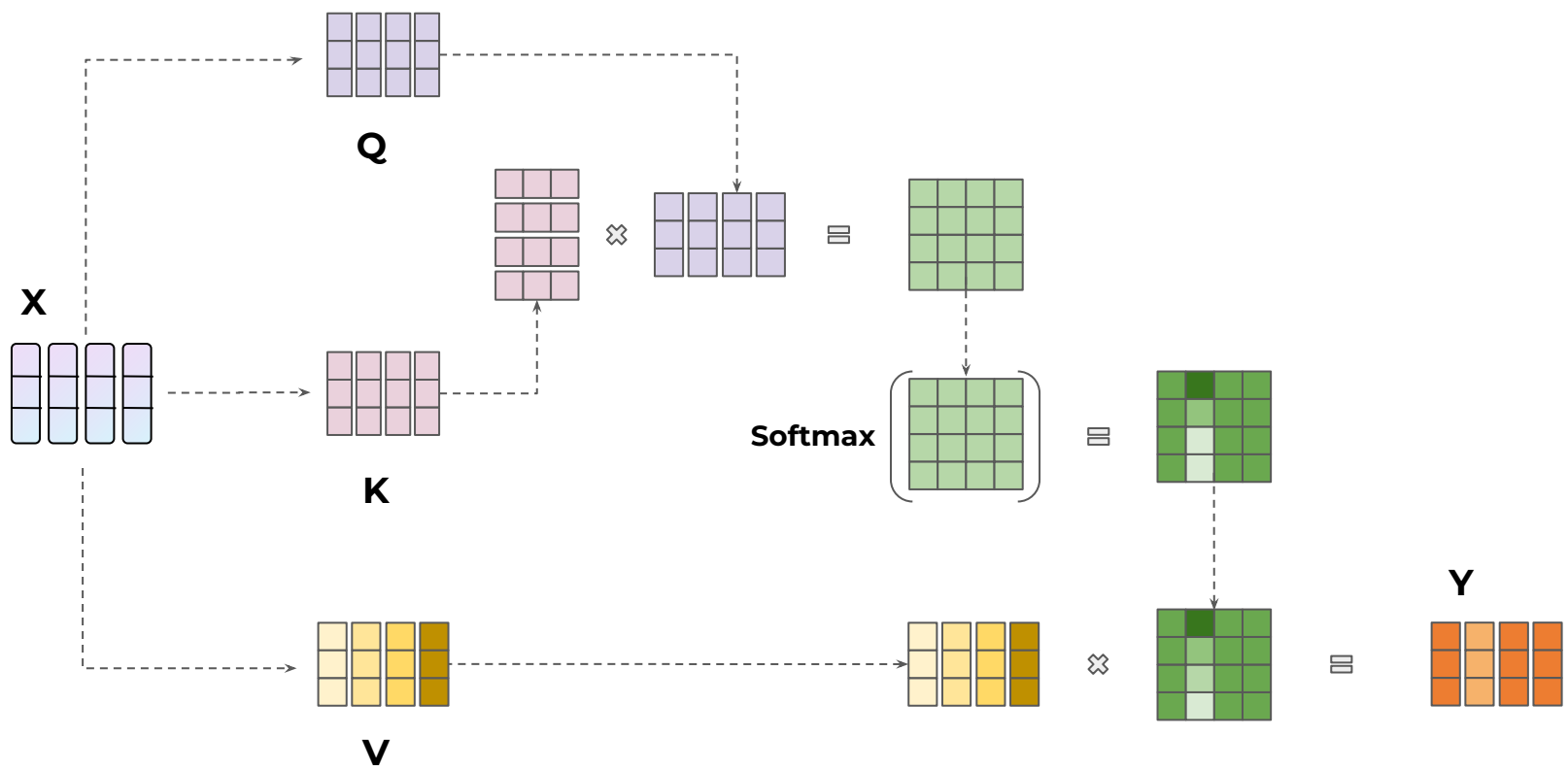
Self-Attention



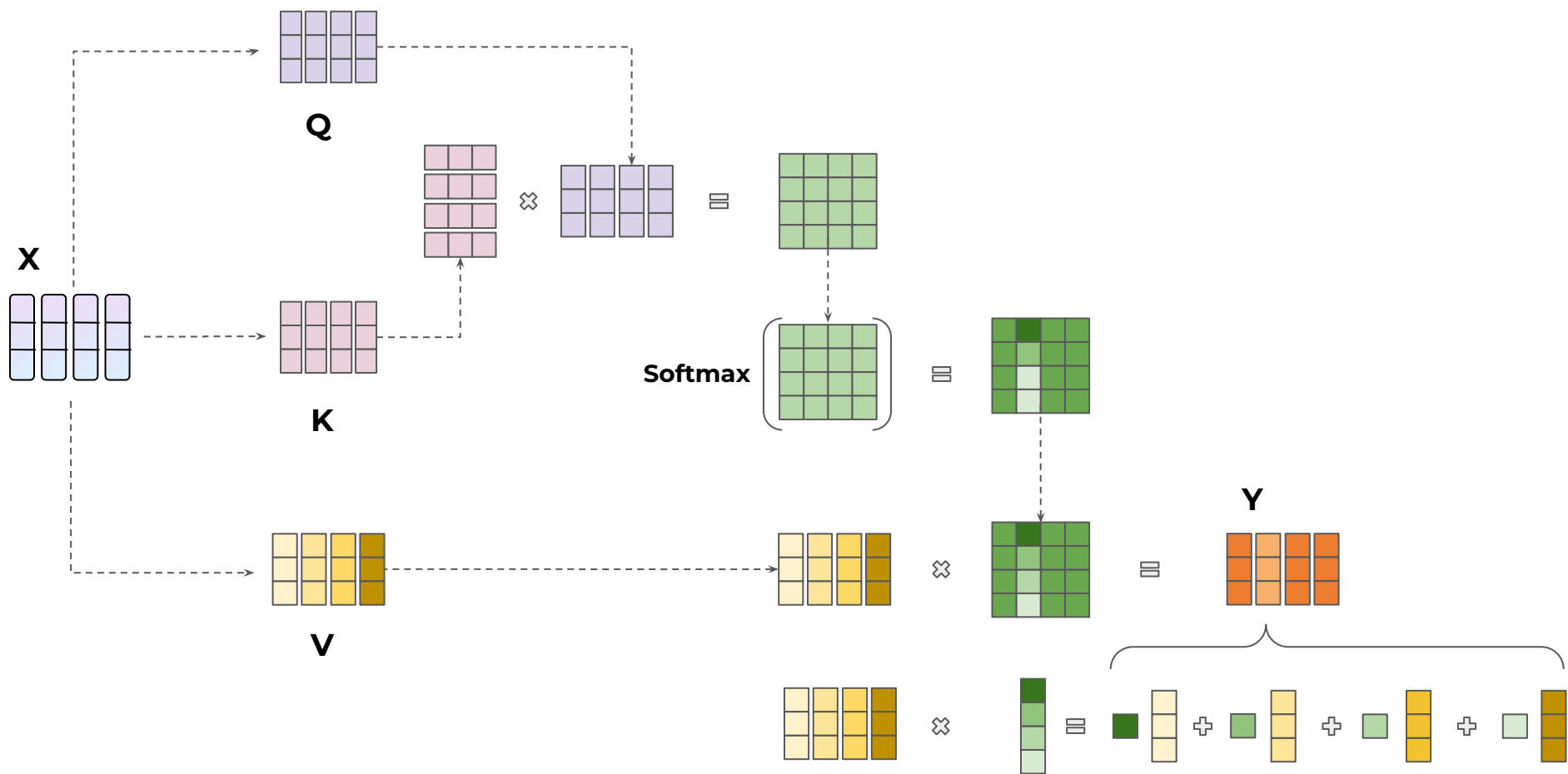
Self-Attention



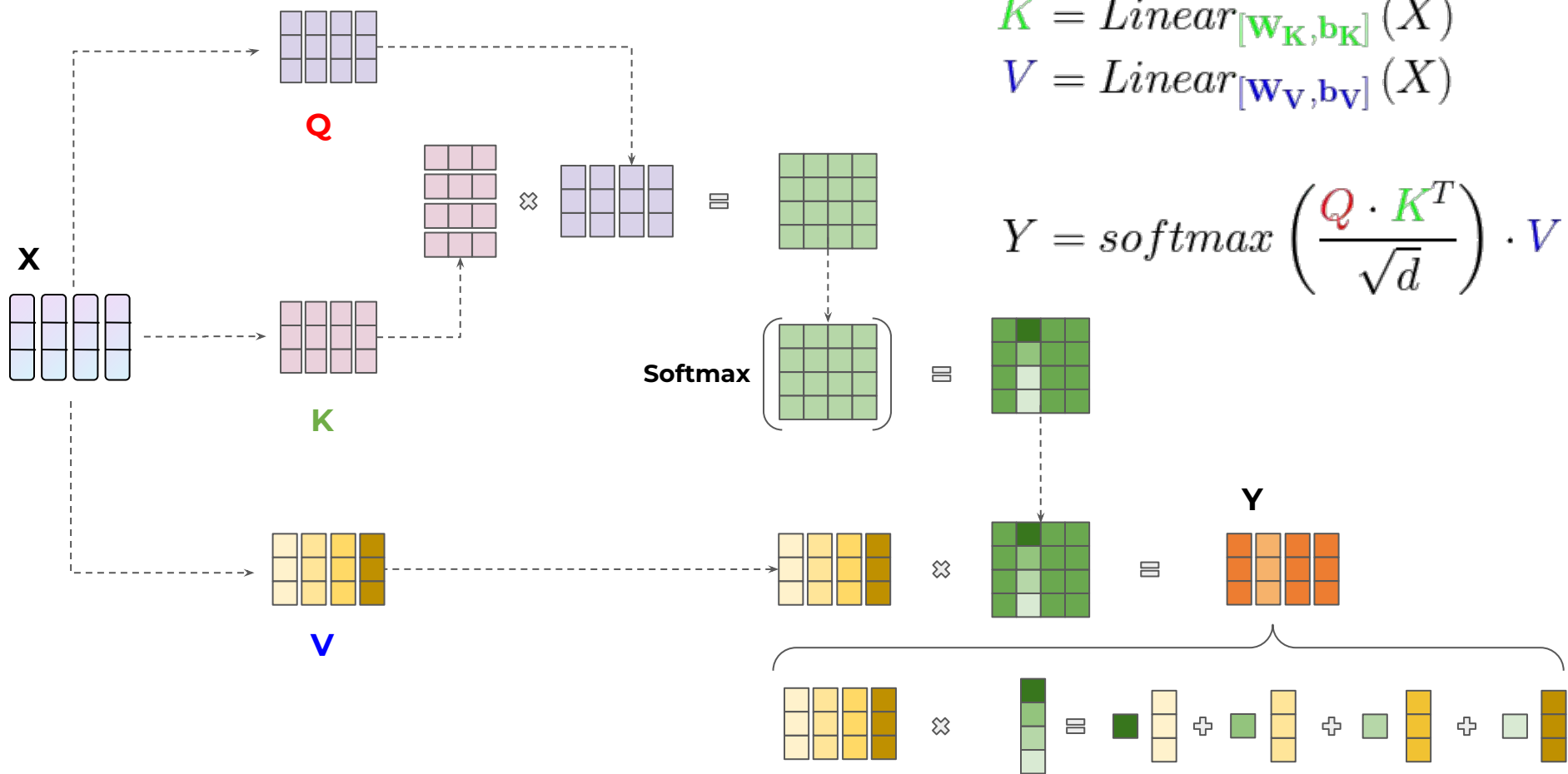
Self-Attention



Self-Attention

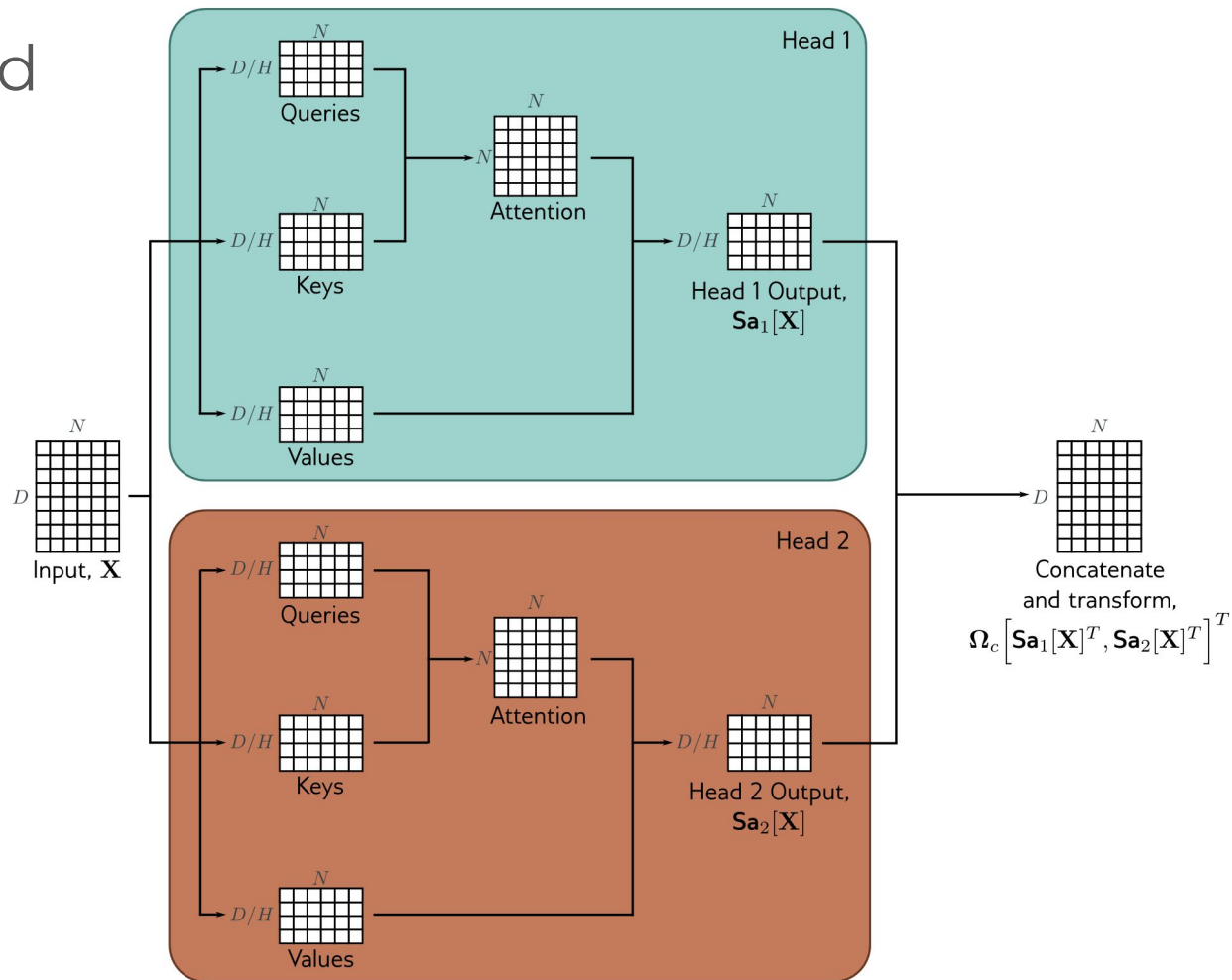


Self-Attention

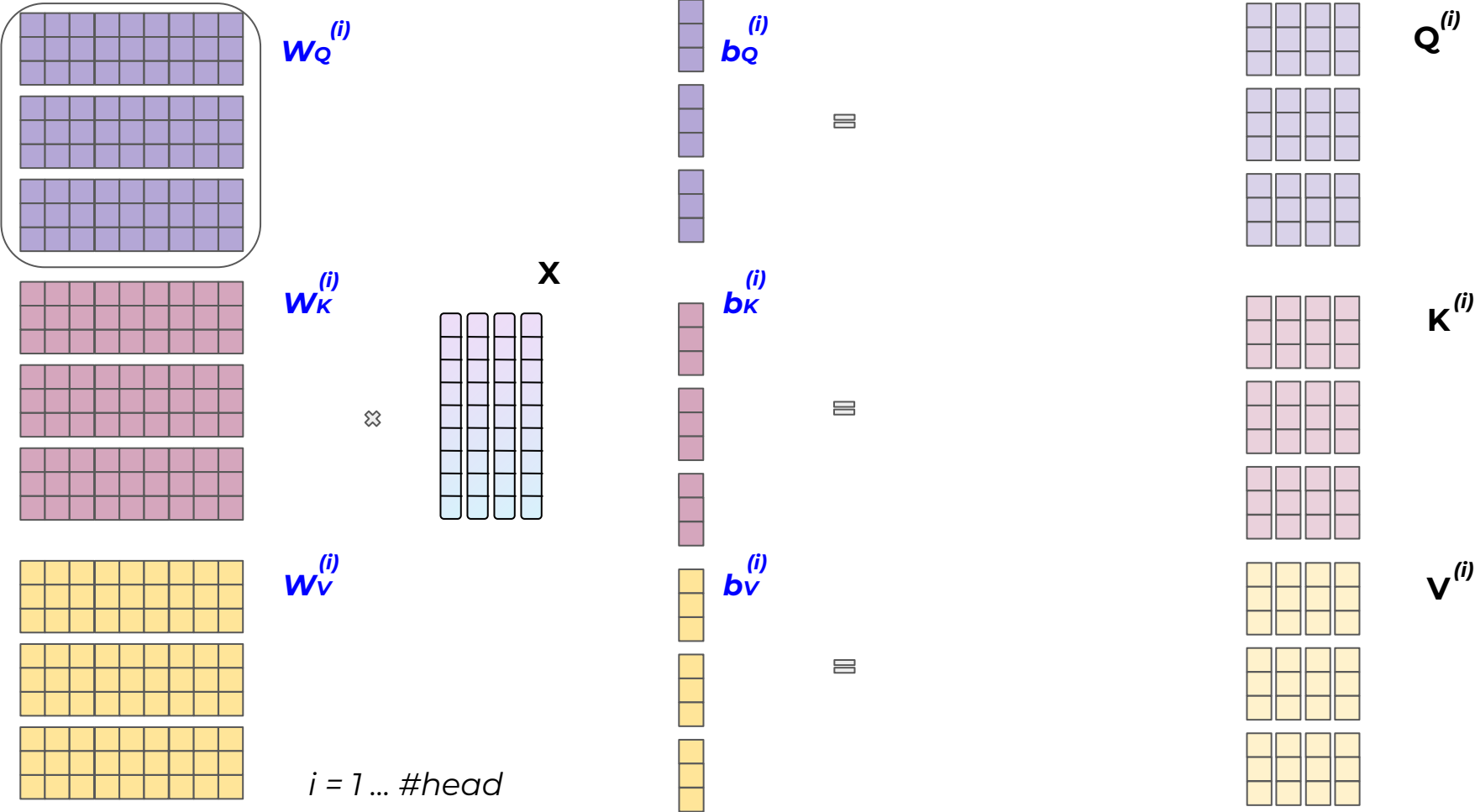


Transformer layer

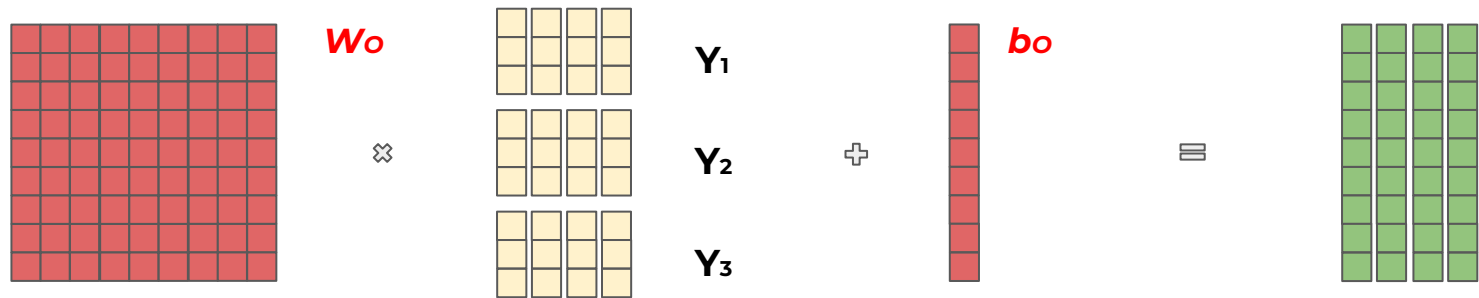
Multihead



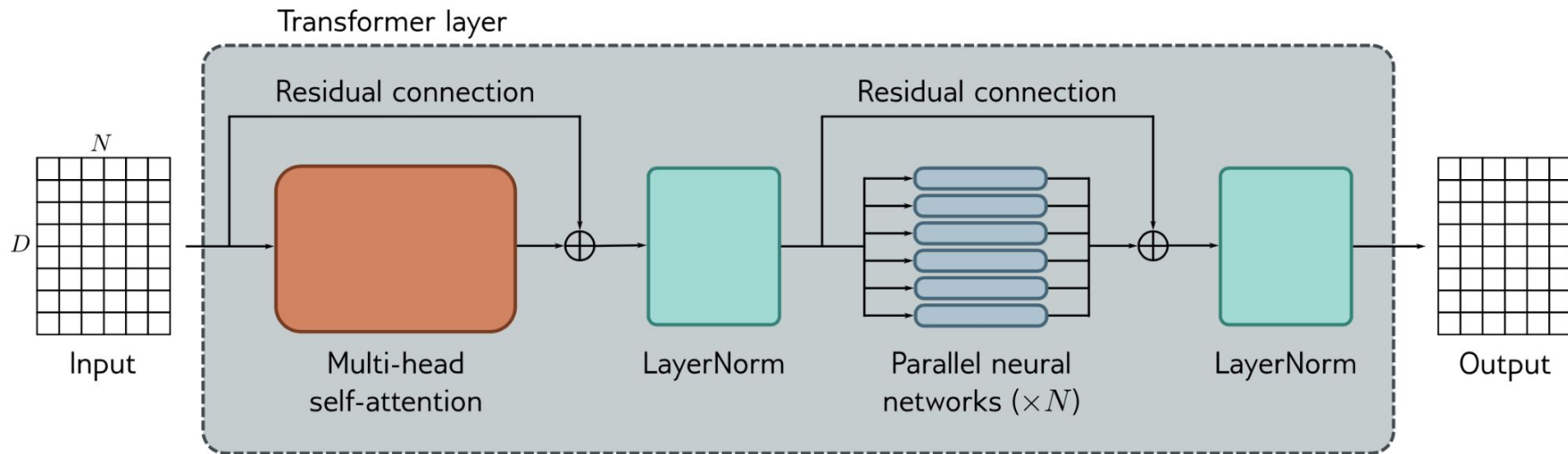
Transformer [Multi-Head Attention (MHA)]



Transformer [Multi-Head Attention (MHA)]

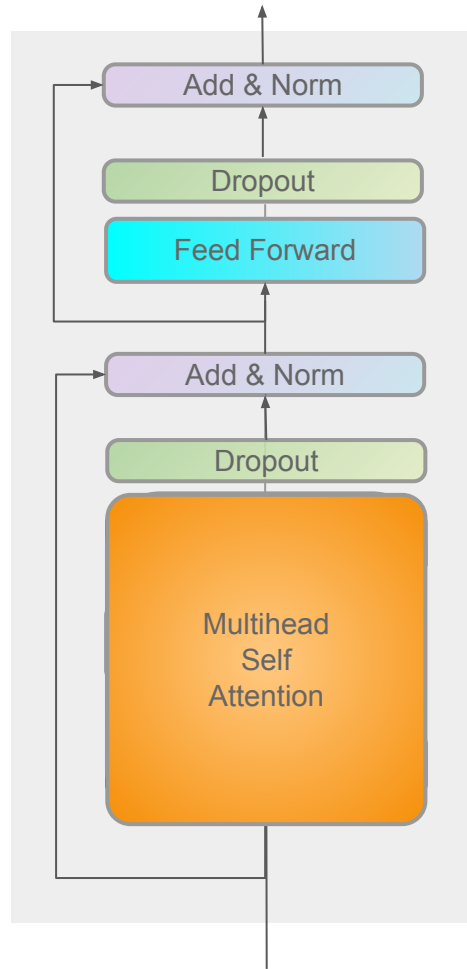


Transformer layer

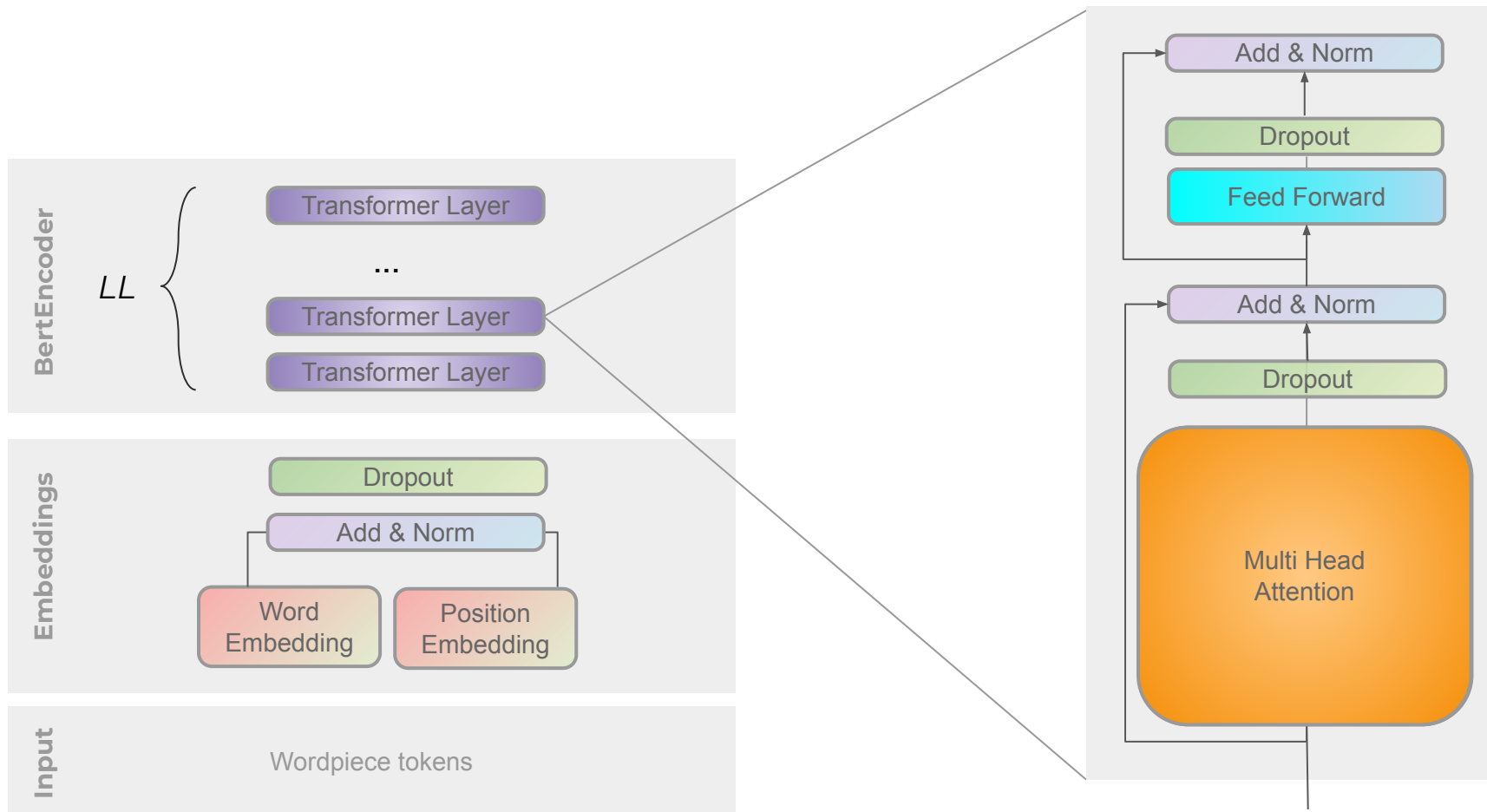


* image from [Understanding Deep Learning](#), book by Simon J.D. Prince, 2023

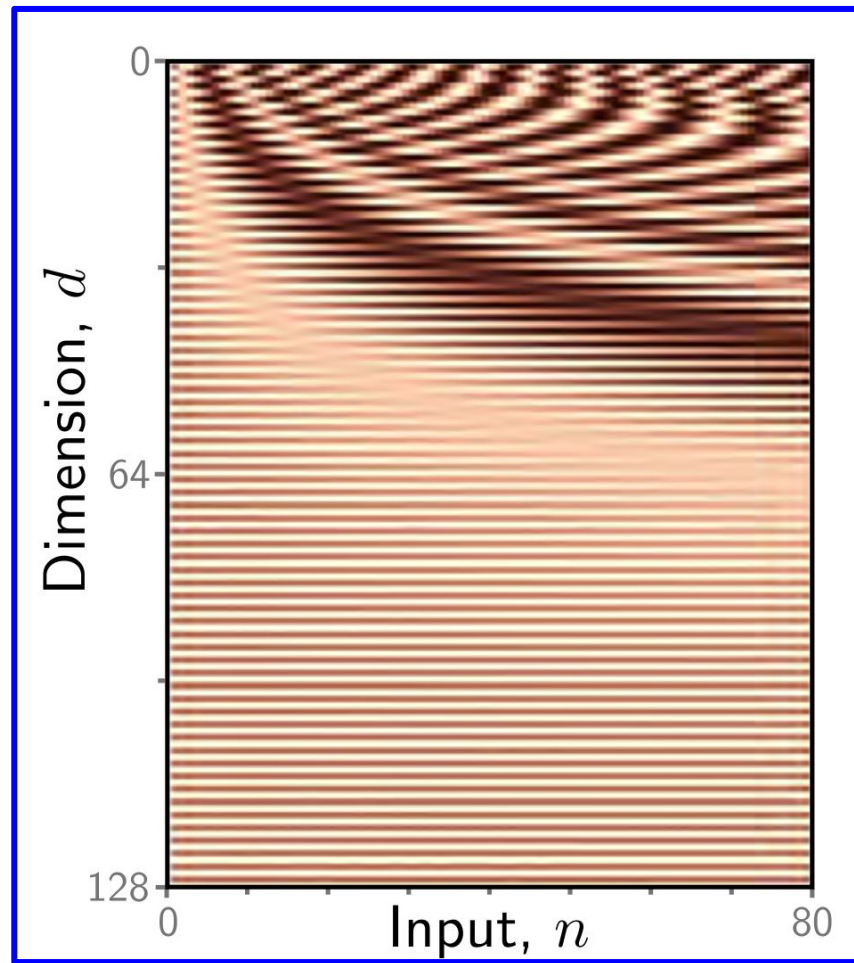
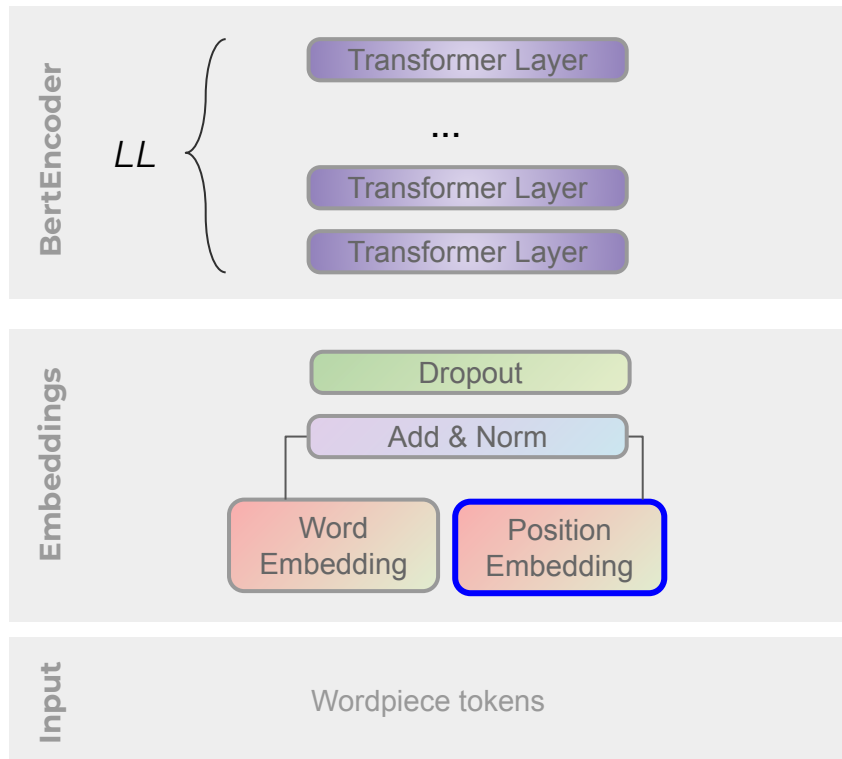
Transformer layer



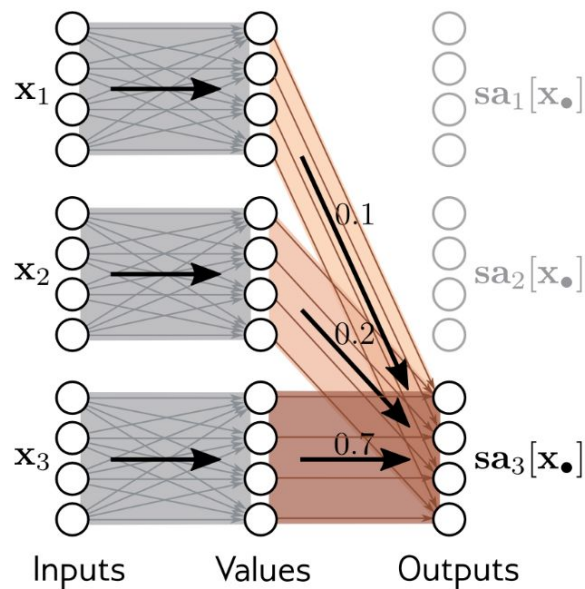
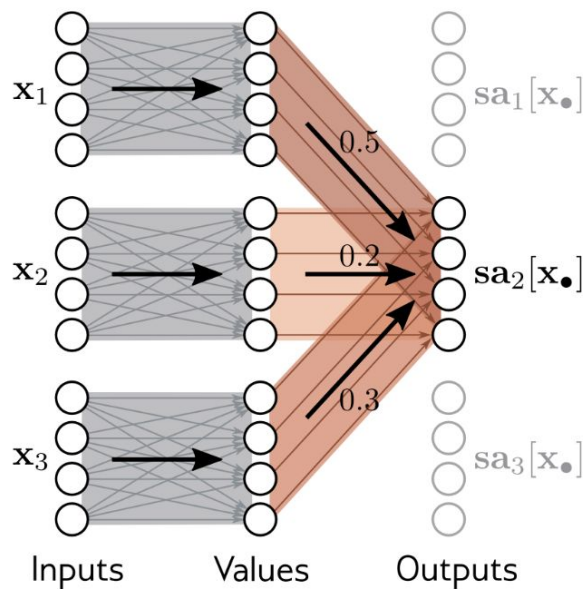
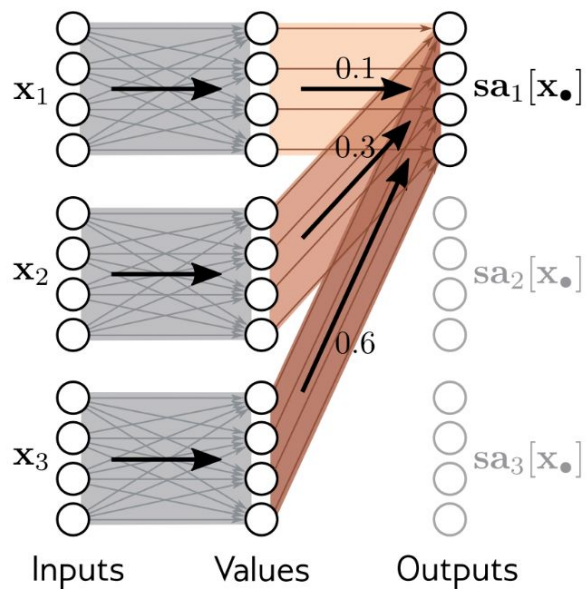
Transformer



Transformer



A bit different representation



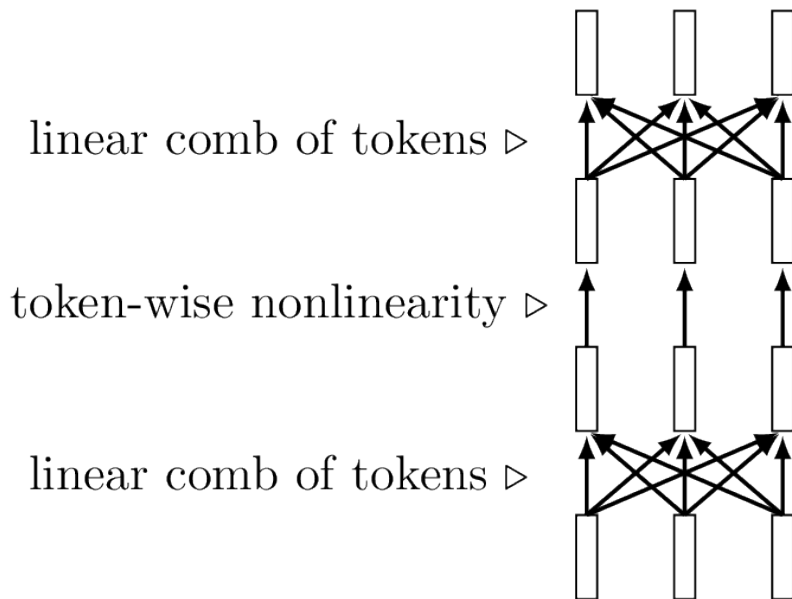
* image from [Understanding Deep Learning](#), book by Simon J.D. Prince, 2023

A bit different representation

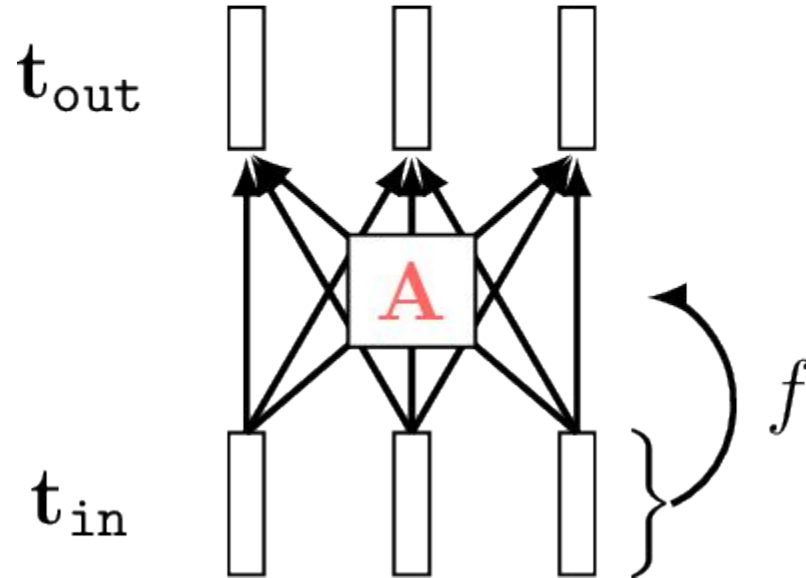
Transformers consist of two main operations over tokens:

- (1) mixing tokens via a weighted sum, and
- (2) modifying each individual token via a nonlinear transformation.

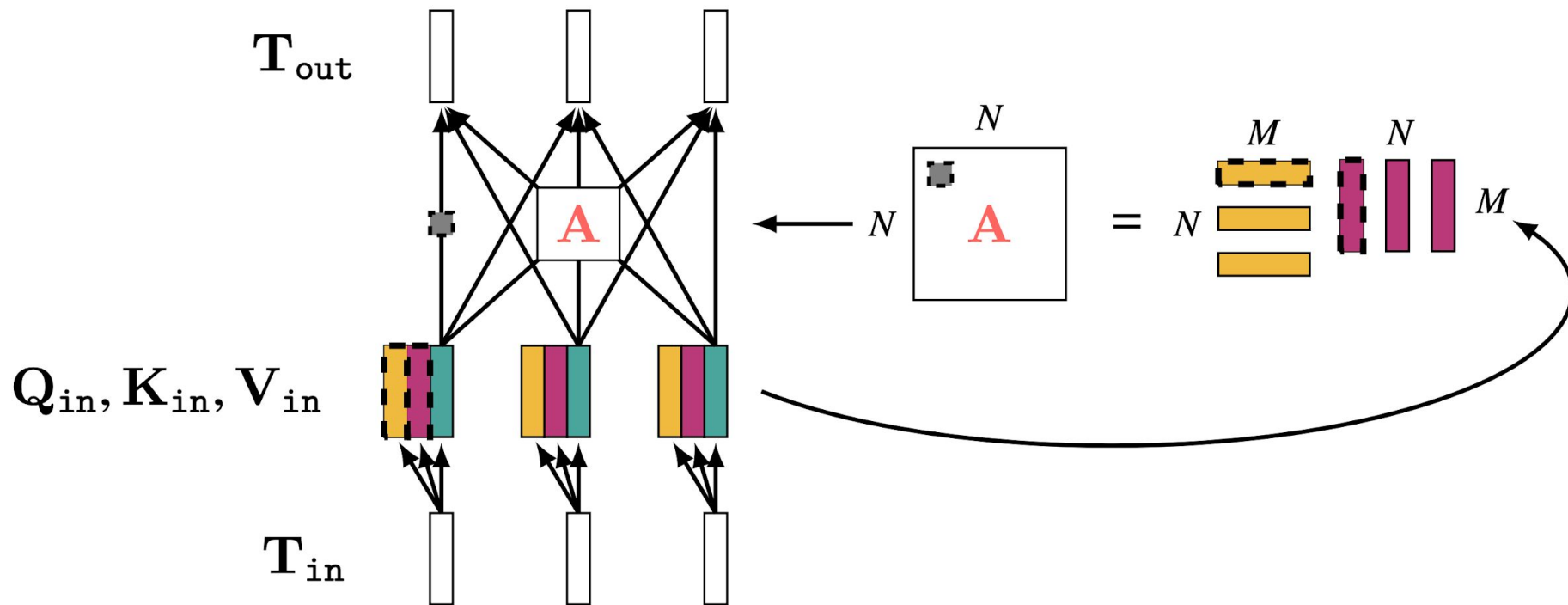
These operations are analogous to the two workhorses of regular neural nets: the linear layer and the pointwise nonlinearity.



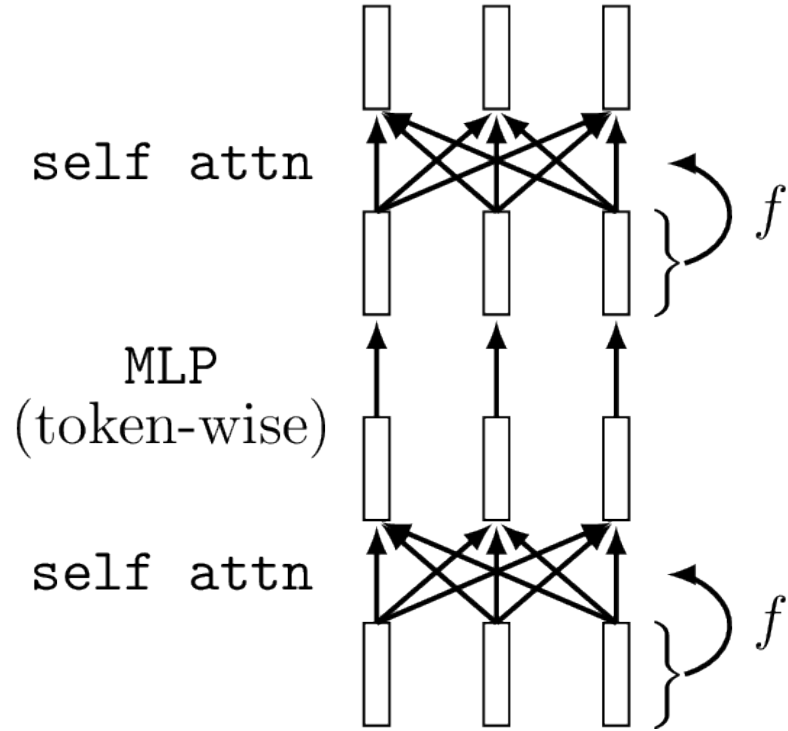
attn layer



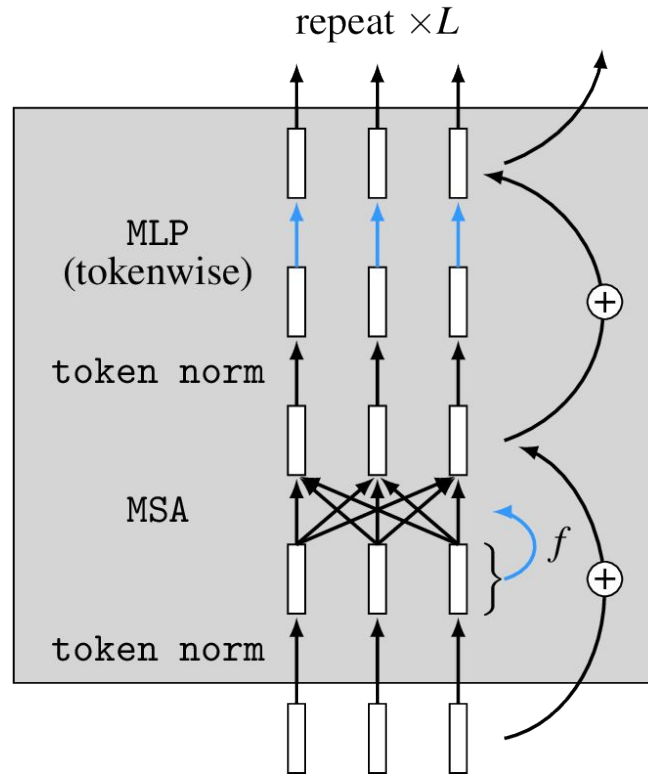
self attn layer (expanded)

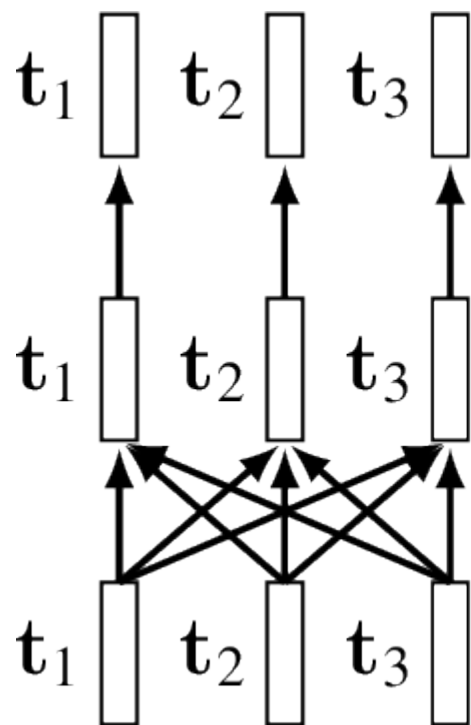


Transformer (vanilla)

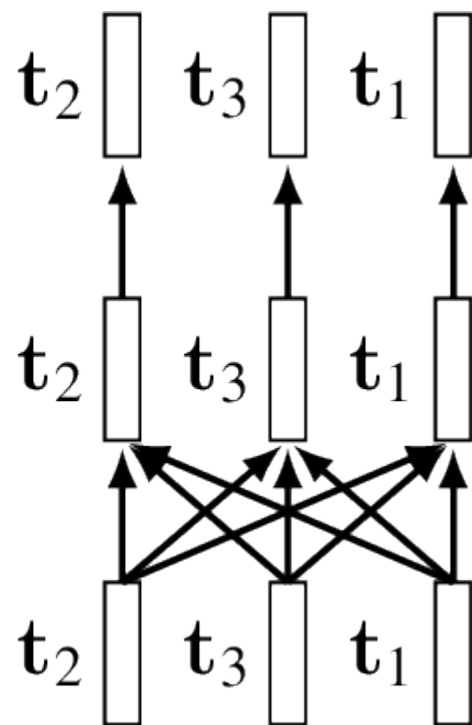


Transformer (ViT) [2]





— permute →

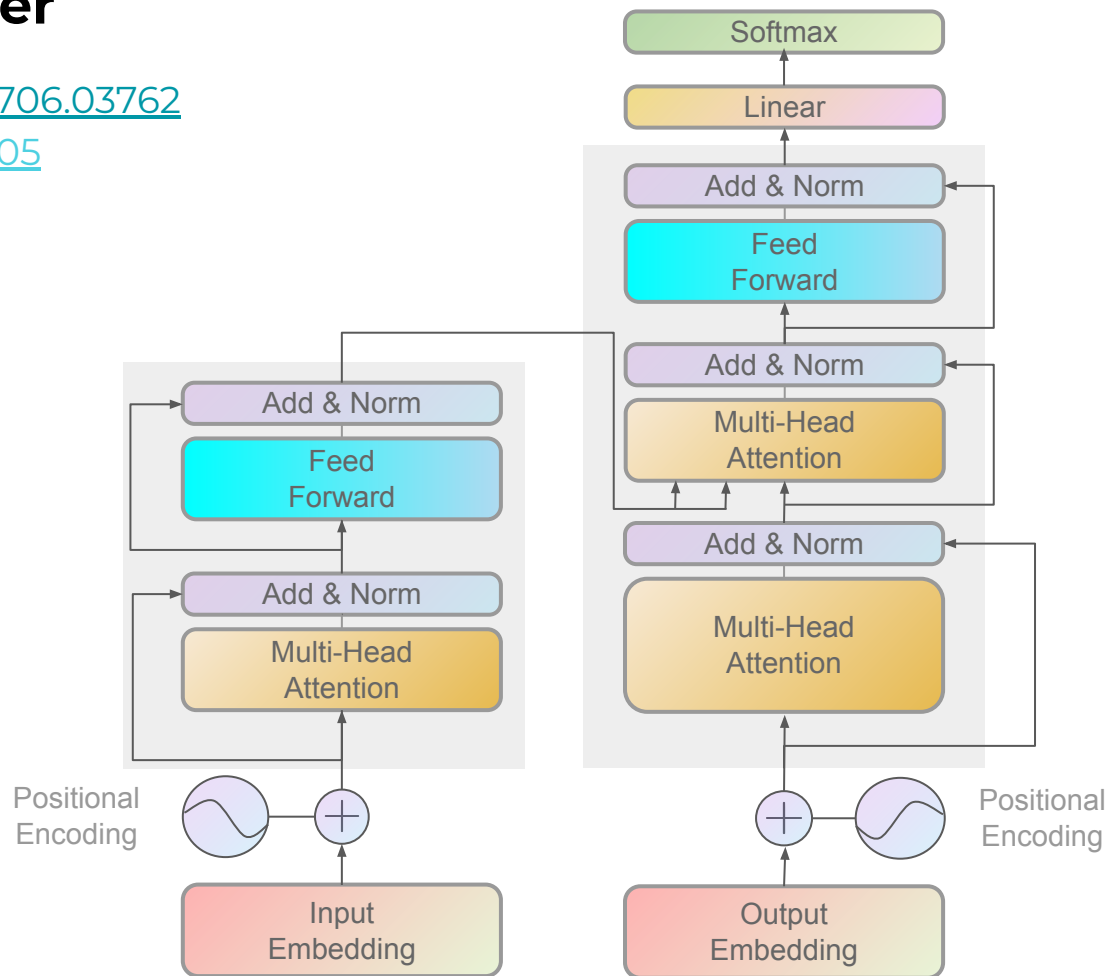


Transformer

Transformer: [1706.03762](#)

BERT: [1810.04805](#)

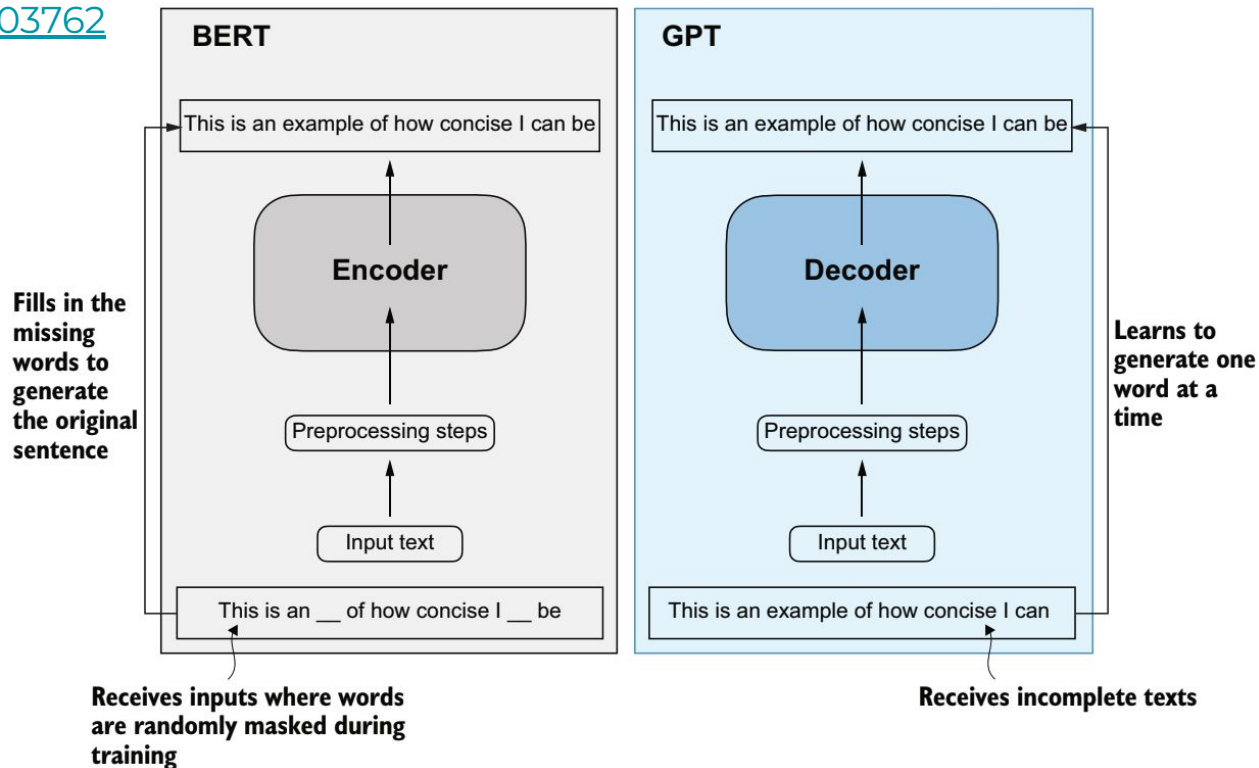
GPT: [1810](#)

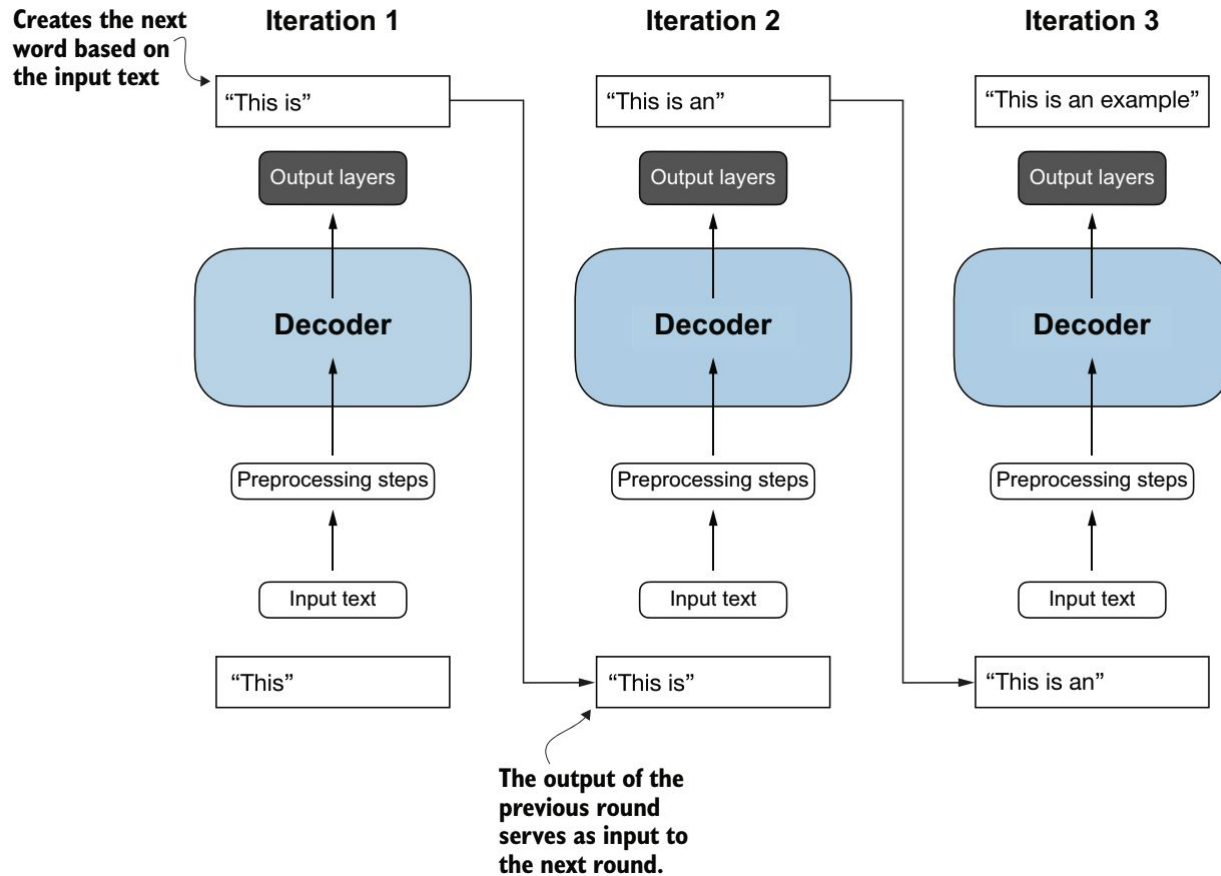


Transformer: [1706.03762](#)

BERT: [1810.04805](#)

GPT: [1810](#)





Memory & Compute

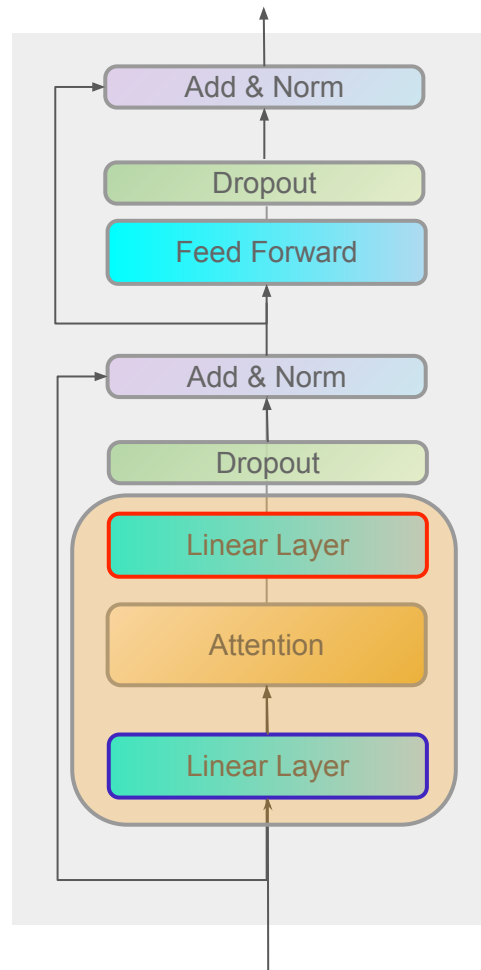
Transformer [Multi head attention]

$$FC(X) = Linear_{[W_{out}, b_{out}]}(\sigma(Linear_{[W_{in}, b_{in}]}(X)))$$

$$MultiHead(\mathbf{X}, \mathbf{X}) = Linear_{[W^o, b^o]} \left(\underset{i \in [A]}{concat} [\mathbf{H}^i] \right)$$

$$Attention(Q, K, V) = softmax \left(\frac{Q \cdot K^T}{\sqrt{d}} \right) \cdot V$$

$$\begin{aligned} \mathbf{H}^i = & Attention(Linear_{[W_Q^{(i)}, b_Q^{(i)}]}(X), \\ & Linear_{[W_K^{(i)}, b_K^{(i)}]}(X), \\ & Linear_{[W_V^{(i)}, b_V^{(i)}]}(X)) \end{aligned}$$



Transformer [Embeddings]

Memory

Computations

$$2 * [H]$$

$$B * L * H$$

$$[V \times H]$$

-

$$[M \times H]$$

-

$$[T \times H]$$

-

B - batch_size

H - hidden

V - vocabulary size

M - max token length

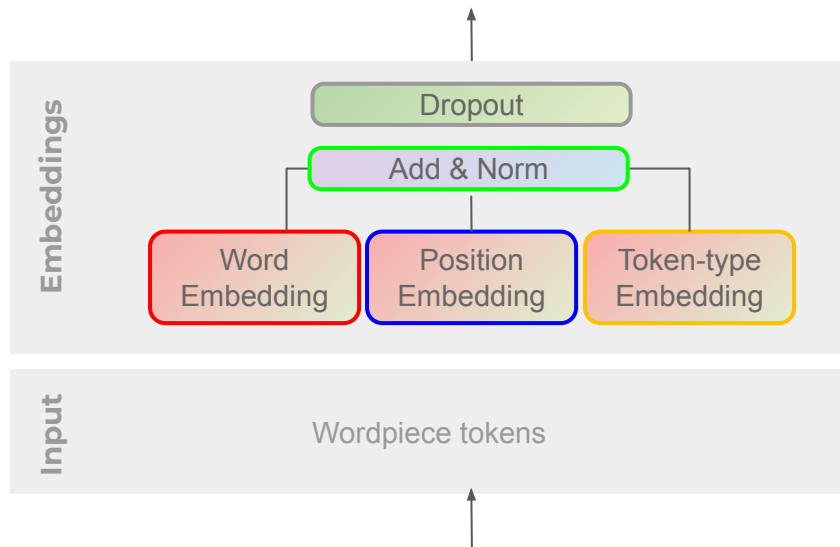
T - number of types

[bert-base: 768]

[bert-base: 50265]

[bert-base: 514]

[bert-base: 1]



Transformer [MHA]

Memory

$$2 * [H]$$

$$[A * S + 1 \times H]$$

-

$$3 * [H + 1 \times A * S]$$

Computations

$$B * L * H$$

$$B * L * (A * S) * H$$

$$B * A * L * L * S * 2$$

$$B * L * H * (A * S) * 3$$

B - batch_size

L - batch_length

H - hidden

A - num_heads

S - head_size (= H/A)

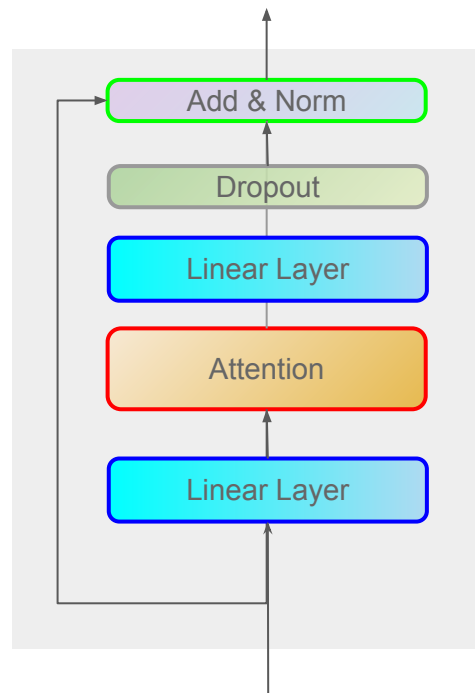
I - intermediate FC size

[bert-base: 768]

[bert-base: 12]

[bert-base: 64]

[bert-base: 3072]



Transformer [FC]

Memory

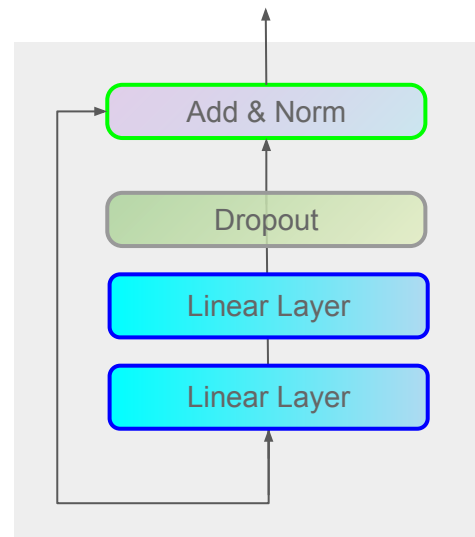
$$2 * [H]$$

$$[H + 1 \times I] + [I + 1 \times H]$$

Computations

$$B * L * H$$

$$2 * B * L * H * I$$



B - batch_size

L - batch_length

H - hidden

A - num_heads

S - head_size (= H/A)

I - intermediate FC size

[bert-base: 768]

[bert-base: 12]

[bert-base: 64]

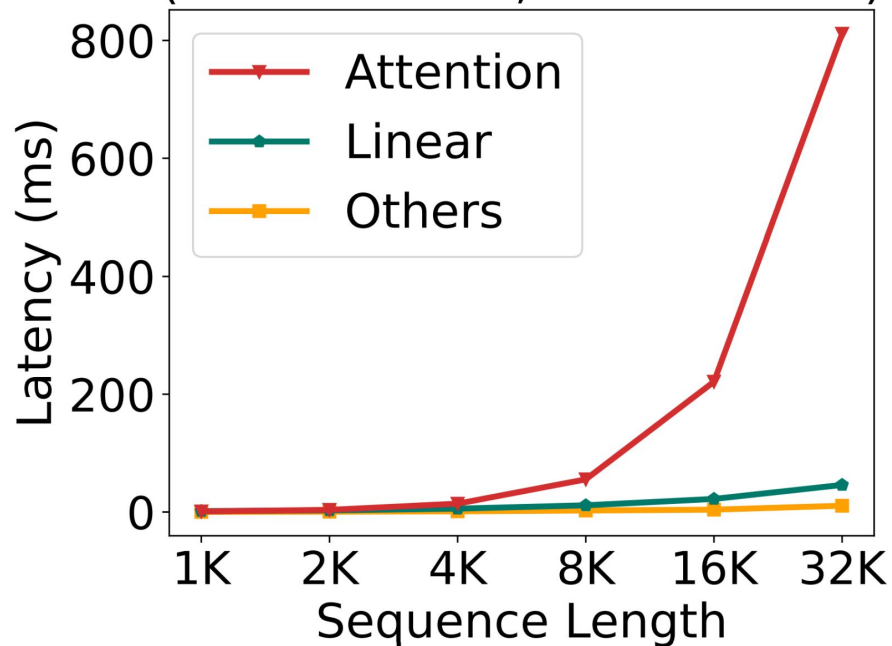
[bert-base: 3072]

Transformer [summary]

		<u>#params [M]</u>	<u>mocs [B]</u>
LL	FC-LayerNorm	$2 * H$	$B * L * H$
	FC-Out	$(1 + H) * I$	$B * L * H * I$
	FC-In	$(1 + I) * H$	$B * L * H * I$
	Atten-LayerNorm	$2 * H$	$B * L * H$
	Atten-LinearOutput	$(1 + H) * H$	$B * L * H * H$
	Atten-AttenScoreValue	0	$B * A * \mathbf{L} * \mathbf{L} * S$
	Atten-AttenScore	0	$B * A * \mathbf{L} * \mathbf{L} * S$
	Atten-LinearInput	$(1 + H) * H * 3$	$B * L * H * H * 3$
	Embedding-LayerNorm	$2 * H$	$B * L * H$
	Embedding-Word	$V * H$	0
	Embedding-Position	$M * H$	0
	Embedding-Token-Type	$T * H$	0

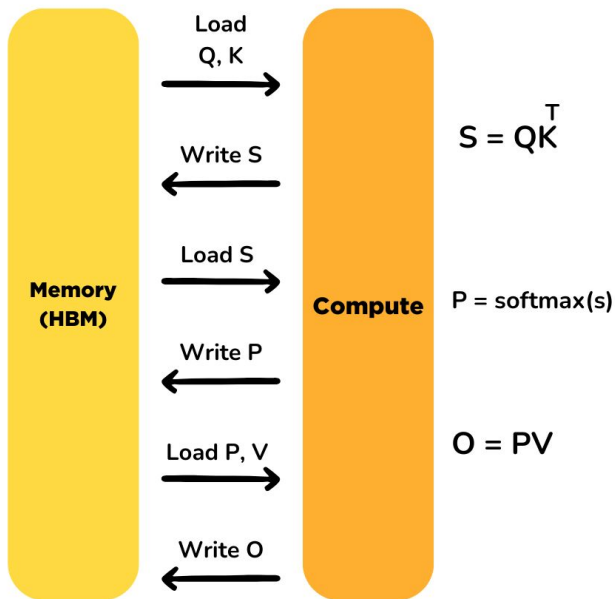
Transformer [Memory & Runtime]

Latency of operators in a transformer model
(NumHead=32, HeadDim=64)

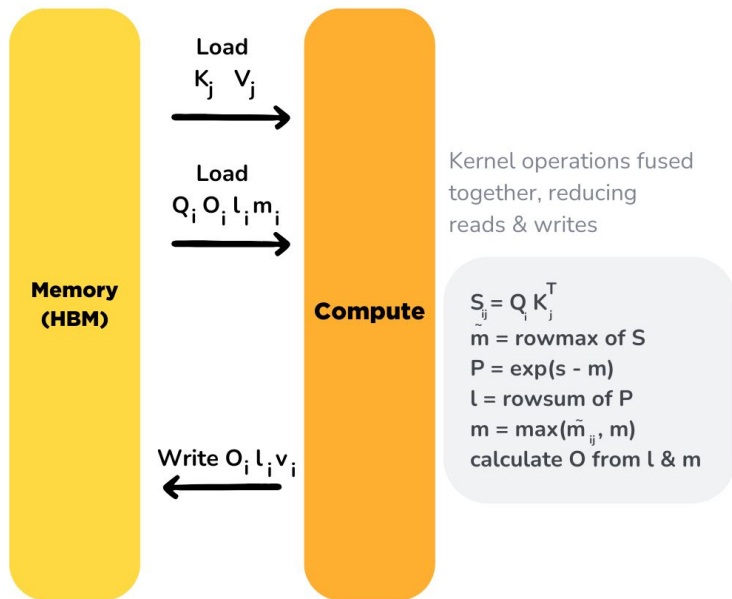


Transformer [Flash Attention]

Standard Attention Implementation



Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Transformer [Flash Attention]

